

Telecom NOC Agentic Copilot

Complete Source Code

Autonomous multi-agent AI for NOC incident response
Qwen3-Coder-30B - AMD Instinct MI300X - 100% on-prem

Files included:

- README.md
- config.py
- settings.py
- llm.py
- pipeline.py
- app.py
- itsm_board.py
- admin_panel.py
- run_all.sh
- data/alarm_generator.py
- data/topology.json
- mocks/itsm_api.py
- mocks/router_api.py
- rag/knowledge_base.py
- runtime_settings.json

README.md

Telecom NOC Agentic Copilot

An autonomous, multi-agent AI system that monitors a telecom Network Operations Center (NOC), detects incidents from a storm of alarms, finds the root cause, recommends and executes remediation (with a human approval gate for risky actions), opens and manages ITSM tickets, and notifies the right teams ? all running **100% on-prem** on a single **AMD Instinct MI300X** GPU with a locally-served **Qwen3-Coder-30B** model.

What it does

When a network outage occurs, a NOC team is flooded with dozens of alarms and must manually correlate them, diagnose the cause, fix it, and raise tickets ? slow, error-prone work that risks SLA breaches. This Copilot does it end-to-end in seconds:

- Ingests** alarms, logs and KPIs from network devices
- Correlates** the alarm storm into a single root incident
- Finds the root cause** using RAG over runbooks + an LLM reranker
- Recommends remediation** with risk levels; auto-executes safe steps, holds risky ones for human approval
- Opens an ITSM ticket** (ServiceNow-style) and keeps it updated through its lifecycle
- Alerts** the right teams based on severity and customer impact

A built-in **Copilot chat** lets an engineer ask questions and drive ticket updates/resolution in natural language.

The multi-agent system

All agents call the same local Qwen model with specialised prompts, orchestrated by ``pipeline.py``:

Agent	Role
Correlation Agent	Collapses ~33 raw alarms across 17 devices into one root incident
RAG + Reranker	Retrieves candidate runbooks (ChromaDB + all-MiniLM-L6-v2), then an LLM reranks them so the best match leads
Root Cause Agent	Diagnoses the underlying fault with a confidence score and evidence
Remediation Agent	Produces a risk-ranked action plan; low-risk steps auto-execute, high-risk steps require approval
Execution	Calls mock device APIs (clear BGP, restart interface, reset router)
ITSM Agent	Creates/updates tickets through a ServiceNow-style board
Alerting Agent	Decides who to notify (on-call, NOC tier-2, management) and drafts the messages

Architecture & services

Service	Port	What it is
vLLM (Qwen3-Coder-30B)	8000	Local LLM inference server
Mock Router API	8001	Simulated device remediation endpoints
Mock ITSM API	8002	In-memory ServiceNow-style ticket store
ITSM Board	8080	Standalone ServiceDesk-style incident board (clickable, live)
Admin Panel	8090	Configure SLA timer + notification recipients
Dashboard	7860	Main command-center UI (Gradio) + Copilot chat

Pipeline flow: alarms ? correlate ? RAG+rerank ? root cause ? remediate ? execute ? ticket ? alert

Tech stack

- Model:** Qwen/Qwen3-Coder-30B-A3B-Instruct (served locally via vLLM 0.11)
- Hardware:** AMD Instinct MI300X (192 GB), ROCm 7.0
- RAG:** ChromaDB vector store + `all-MiniLM-L6-v2` sentence embeddings + LLM reranking
- Backend:** Python 3.12, FastAPI (mock APIs + board + admin), Gradio (dashboard)
- Everything runs on-prem** ? no external API calls, full data sovereignty

Prerequisites

- An AMD Instinct MI300X (or compatible ROCm GPU) with vLLM installed
- Python 3.12
- Python packages: `gradio`, `fastapi`, `uvicorn`, `requests`, `chromadb`, `sentence-transformers`, `openai` (client), plus the pinned dependency below

****Important dependency pin**** (RAG needs this exact range, or retrieval silently fails):

```
```bash
pip install "huggingface-hub>=0.34.0,<1.0"
```
```

How to run (step by step)

1. Start the local LLM (vLLM) ? Terminal 1

```
```bash
vllm serve Qwen/Qwen3-Coder-30B-A3B-Instruct \
 --host 0.0.0.0 --port 8000 \
 --tensor-parallel-size 1 \
 --max-model-len 100000 \
 --enable-auto-tool-choice \
 --tool-call-parser qwen3_xml \
 --disable-log-stats
```
```

Wait until it prints that the server is running. Verify:

```
```bash
curl -s http://localhost:8000/v1/models
```
```

You should see `Qwen/Qwen3-Coder-30B-A3B-Instruct`.

2. Start all app services ? Terminal 2

A single script starts the mock APIs, the ITSM board, the admin panel, and the dashboard (it leaves vLLM alone, frees any stale ports, health-checks each service, and prints the URLs):

```
```bash
bash run_all.sh
```
```

Healthy output shows `router :8001 OK`, `itsm :8002 OK`, `board :8080 OK`,
`admin :8090 OK`, then a public dashboard link.

3. Open the interfaces

- ****Dashboard:**** the `gradio.live` link printed by `run\_all.sh` (or port 7860 via your proxy)
- ****ITSM Board:**** `- **\*\*Admin Panel:\*\*** `

> If you run behind a different host/proxy, edit `PROXY\_BASE` at the top of `app.py`
> so the in-app links to the ITSM board and admin panel point to the right place.

4. Use it

- Click ****? Simulate P1 Outage (Critical)**** to run the full pipeline on a core-router incident
- Or ****? Simulate Random Incident**** for a varied-severity scenario (P2/P3)
- Watch the dashboard fill in: alarms ? correlation ? RAG ? root cause ? remediation ? ticket ? notifications
- Click ****? Approve & Execute High-Risk Action**** to authorise the risky step (router reset)
- Open the ****ITSM Board**** to see the live ticket; click it for full details and to resolve it
- Use the ****Copilot chat**** to ask questions or update/resolve tickets (e.g. "resolve INC123 and add a note")
- Open the ****Admin Panel**** to change the SLA timer or notification teams ? changes reflect live

To stop / restart

Press ****Ctrl+C**** in the `run\_all.sh` terminal, then run `bash run\_all.sh` again.
Leave vLLM (Terminal 1) running ? no need to restart the model.

Troubleshooting

- ****RAG shows "Awaiting retrieval" / board empty:**** re-pin the dependency and restart:
`pip install "huggingface-hub>=0.34.0,<1.0"` then `bash run\_all.sh`
- ****vLLM won't start ("free memory less than...")**** a previous vLLM is still holding the GPU.
Find it with `rocm-smi`, kill the old process, then start vLLM again.
- ****"address already in use:"**** `run\_all.sh` already frees the app ports; if it persists,
`fuser -k -9 8001/tcp 8002/tcp 8080/tcp 8090/tcp 7860/tcp` then re-run.

Project structure

...

| | |
|-----------------------|---|
| app.py | Main dashboard (Gradio) + Copilot chat + approval flow |
| pipeline.py | The multi-agent orchestrator (correlate ? ... ? alert) |
| llm.py | Local LLM client + JSON helpers + per-agent token logging |
| config.py | Model name, ports, URLs, SLA default |
| settings.py | Runtime settings store (SLA, notification recipients) |
| itsm_board.py | Standalone ServiceNow-style ITSM board (port 8080) |
| admin_panel.py | Admin / configuration panel (port 8090) |
| run_all.sh | One command to start all app services |
| runtime_settings.json | Persisted admin settings |
| data/ | |
| alarm_generator.py | Generates the alarm storm + scenario variants |
| topology.json | 17-device network topology |
| mocks/ | |
| router_api.py | Mock device remediation API (port 8001) |
| itsm_api.py | Mock ITSM ticket API (port 8002) |
| rag/ | |
| knowledge_base.py | ChromaDB + embeddings runbook retrieval |
| ... | |

Notes for reviewers

- Input alarms are **synthetic** (generated to simulate a realistic outage); the **AI** processing is **real** ? every agent makes a live call to the local model.
- The ITSM board is **styled like ServiceNow**; in production this would connect to a real ServiceNow instance via the same update API.
- Token usage per agent is logged to the dashboard terminal for transparency on efficiency.

config.py

```
# config.py - Central configuration for the NOC Copilot system

VLLM_BASE_URL = "http://localhost:8000/v1"
MODEL_NAME = "Qwen/Qwen3-Coder-30B-A3B-Instruct"
API_KEY = "EMPTY"

# Service ports
ROUTER_API_PORT = 8001
ITSM_API_PORT = 8002
DASHBOARD_PORT = 7860

ROUTER_API_URL = f"http://localhost:{ROUTER_API_PORT}"
ITSM_API_URL = f"http://localhost:{ITSM_API_PORT}"

# Paths
CHROMA_DB_PATH = "./chroma_db"
TOPOLOGY_PATH = "data/topology.json"
RUNBOOKS_PATH = "data/runbooks"

# SLA (seconds) - the breach window for the demo
SLA_SECONDS = 300
```

settings.py

```
# settings.py - Runtime-configurable settings store (JSON-backed).
import json, os, threading
_LOCK = threading.Lock()
_HERE = os.path.dirname(os.path.abspath(__file__))
SETTINGS_FILE = os.path.join(_HERE, "runtime_settings.json")
DEFAULTS = {
    "sla_seconds": 300,
    "notification_recipients": [
        {"team": "On-call Network Engineer", "channel": "page", "contact": "oncall@telco.example"},
        {"team": "NOC Tier-2", "channel": "slack", "contact": "#noc-tier2"},
        {"team": "Engineering Manager", "channel": "email", "contact": "eng-manager@telco.example"},
    ],
}

def load_settings():
    s = dict(DEFAULTS)
    try:
        if os.path.exists(SETTINGS_FILE):
            with open(SETTINGS_FILE) as f:
                s.update(json.load(f))
    except Exception:
        pass
    return s

def save_settings(new_settings):
    with _LOCK:
        cur = load_settings(); cur.update(new_settings)
        with open(SETTINGS_FILE, "w") as f:
            json.dump(cur, f, indent=2)
    return cur

def get_sla_seconds():
    return int(load_settings().get("sla_seconds", 300))

def get_recipients():
    return load_settings().get("notification_recipients", [])

def add_recipient(team, channel, contact):
    s = load_settings(); recips = s.get("notification_recipients", [])
    recips.append({"team": team, "channel": channel, "contact": contact})
    return save_settings({"notification_recipients": recips})

def remove_recipient(index):
    s = load_settings(); recips = s.get("notification_recipients", [])
    if 0 <= index < len(recips): recips.pop(index)
    return save_settings({"notification_recipients": recips})
```

```
# llm.py - Shared LLM client for the NOC Copilot
# Talks ONLY to the local vLLM endpoint (internal, no external APIs).
# Handles Qwen3-32B's <think>...</think> blocks and robust JSON extraction.

import json
import re
from openai import OpenAI
from config import VLLM_BASE_URL, MODEL_NAME, API_KEY

client = OpenAI(base_url=VLLM_BASE_URL, api_key=API_KEY)

def strip_think(text: str) -> str:
    """Remove <think>...</think> reasoning blocks that Qwen3 emits.

    Handles the normal case, plus a dangling </think> with no opening tag
    (which can happen when generation is cut off).
    """
    if not text:
        return ""
    # Remove well-formed <think>...</think> blocks
    text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL | re.IGNORECASE)
    # If a stray closing tag remains, drop everything before it
    if "</think>" in text:
        text = text.split("</think>")[-1]
    # Drop any leftover lone opening tag
    text = text.replace("<think>", "")
    return text.strip()

def _extract_json_blob(text: str) -> str:
    """Pull the most likely JSON object/array substring out of free text."""
    text = text.strip()
    # Strip markdown code fences if present
    text = re.sub(r"^```(?:json)?", "", text, flags=re.IGNORECASE).strip()
    text = re.sub(r"```$", "", text).strip()

    # Find the outermost { } or [ ]
    first_obj, last_obj = text.find("{"), text.rfind("}")
    first_arr, last_arr = text.find("[", text.rfind("]"))

    candidates = []
    if first_obj != -1 and last_obj != -1 and last_obj > first_obj:
        candidates.append(text[first_obj:last_obj + 1])
    if first_arr != -1 and last_arr != -1 and last_arr > first_arr:
        candidates.append(text[first_arr:last_arr + 1])

    # Prefer the longer candidate (usually the full structure)
    if candidates:
        return max(candidates, key=len)
    return text

def parse_json(text: str):
    """Strip thinking blocks + fences, then parse JSON. One repair retry."""
    clean = strip_think(text)
    blob = _extract_json_blob(clean)
    try:
        return json.loads(blob)
    except json.JSONDecodeError:
        # Repair pass: common LLM JSON mistakes
        repaired = blob
        repaired = re.sub(r",\s*([}\]])", r"\1", repaired) # trailing commas
        repaired = repaired.replace('"', "'") # single quotes
        repaired = re.sub(r"\bNone\b", "null", repaired)
        repaired = re.sub(r"\bTrue\b", "true", repaired)
        repaired = re.sub(r"\bFalse\b", "false", repaired)
        try:
            return json.loads(repaired)
        except json.JSONDecodeError as e:
            raise ValueError(
                f"Could not parse JSON from model output.\n"
                f"--- cleaned text ---\n{clean[:800]}\n--- error ---\n{e}"
            )
```

)

```
TOKEN_USAGE = {"calls": 0, "prompt": 0, "completion": 0, "total": 0, "by_agent": {}}
```

```
def _log_usage(resp, agent="chat"):
    try:
        u = resp.usage
        TOKEN_USAGE["calls"] += 1
        TOKEN_USAGE["prompt"] += u.prompt_tokens
        TOKEN_USAGE["completion"] += u.completion_tokens
        TOKEN_USAGE["total"] += u.total_tokens
        a = TOKEN_USAGE["by_agent"].setdefault(agent, {"calls": 0, "prompt": 0, "completion": 0, "total": 0})
        a["calls"] += 1; a["prompt"] += u.prompt_tokens
        a["completion"] += u.completion_tokens; a["total"] += u.total_tokens
        print(f"[tokens] {agent:14s} prompt={u.prompt_tokens:6d} completion={u.completion_tokens:6d} "
              f"total={u.total_tokens:6d} | session total={TOKEN_USAGE['total']}", flush=True)
    except Exception:
        pass
```

```
def token_summary():
    lines = ["=== TOKEN USAGE (this session) ==="]
    for ag, v in TOKEN_USAGE["by_agent"].items():
        lines.append(f" {ag:14s} {v['calls']:3d} calls {v['total']:7d} tokens "
                    f"(in {v['prompt']}, out {v['completion']})")
    lines.append(f" {'TOTAL':14s} {TOKEN_USAGE['calls']:3d} calls {TOKEN_USAGE['total']:7d} tokens")
    return "\n".join(lines)
```

```
def chat(messages, tools=None, thinking=False, temperature=0.3, max_tokens=4000, agent="chat"):
    """Send a chat request to the local vLLM and return the message object.
```

```
thinking=False is the default: faster and cleaner JSON for agent work.
Pass tools=[...] to enable native tool-calling (Remediation agent).
"""
```

```
extra_body = {"chat_template_kwargs": {"enable_thinking": thinking}}
```

```
kwargs = {
    "model": MODEL_NAME,
    "messages": messages,
    "temperature": temperature,
    "max_tokens": max_tokens,
    "extra_body": extra_body,
}
if tools:
    kwargs["tools"] = tools
    kwargs["tool_choice"] = "auto"
```

```
resp = client.chat.completions.create(**kwargs)
_log_usage(resp, agent)
return resp.choices[0].message
```

```
def ask_json(system_prompt, user_payload, thinking=False, temperature=0.2, agent="chat"):
    """Convenience: send a system + user(JSON) message, return parsed JSON dict.
```

```
user_payload may be a dict (auto-serialised) or a string.
"""
```

```
if isinstance(user_payload, (dict, list)):
    user_content = json.dumps(user_payload, indent=2)
else:
    user_content = str(user_payload)
```

```
msg = chat(
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_content},
    ],
    thinking=thinking,
    temperature=temperature,
    agent=agent,
)
return parse_json(msg.content)
```



```
if __name__ == "__main__":
    # Foundation test: prove think-stripping + JSON parsing round-trips cleanly.
    print("Test 1: thinking disabled")
    out = ask_json(
        "You output only JSON. No prose.",
        'Return exactly this JSON: {"status": "ok", "value": 42}',
    )
    print("  Parsed:", out)
    assert out.get("status") == "ok", "unexpected output"

    print("Test 2: strip_think on a synthetic thinking block")
    sample = '<think>Let me think about this carefully...</think>\n{"answer": "clean"}'
    print("  Parsed:", parse_json(sample))
    assert parse_json(sample)["answer"] == "clean"

    print("\nAll foundation tests passed. llm.py is working.")
```

pipeline.py

```
# pipeline.py - The REAL orchestrator. Calls the actual agents in sequence.
# This replaces the fake orchestrator.py whose nodes returned hardcoded data.
#
# It is written as a plain sequential pipeline (no LangGraph dependency needed
# to run) but exposes the same stages. A generator version (run_pipeline_streaming)
# yields after each stage so the dashboard can update live.

import json
import os
import time

from llm import ask_json
from data.alarm_generator import generate_storm, generate_scenario, load_topology

# ---- Tool definitions passed to the remediation agent ----
TOOLS = [
    {"name": "clear_bgp", "description": "Clear/reset a BGP session on a device (low risk).",
     "args": ["device_id", "peer"]},
    {"name": "restart_interface", "description": "Bounce a network interface (low risk).",
     "args": ["device_id", "interface"]},
    {"name": "reset_router", "description": "Reboot a router (HIGH RISK, requires approval).",
     "args": ["device_id"]},
]

# ---- Agent system prompts (kept here so the pipeline is self-contained) ----
SHARED_RULES = """
OUTPUT RULES:
- Respond with ONE valid JSON object and NOTHING else.
- No markdown code fences. No prose before or after. No explanation.
- If unsure, still return valid JSON with your best estimate.
"""

CORRELATION_PROMPT = """You are the Correlation Agent in a Telecom NOC. You receive normalised
alarms that arrived in a short window. Many are symptoms of the SAME underlying fault.
1. Group alarms into incidents using topology, timing and causal patterns (a core router
   failure causes downstream BGP, packet-loss and link-down alarms).
2. Identify the single ROOT EVENT alarm ? earliest, highest in topology, most causal.
3. Collapse aggressively: a 40-alarm storm is usually ONE incident.

Return JSON exactly:
{"incidents":[{"incident_id":"INC-001","root_event_alarm_id":"...", "member_alarm_ids":[...],
"affected_devices":[...], "severity":"CRITICAL", "correlation_reason":"one sentence", "alarm_count":N}]}
""" + SHARED_RULES

ROOT_CAUSE_PROMPT = """You are the Root Cause Analysis Agent, a senior telecom network engineer.
Given one correlated incident, its alarms, and runbook excerpts, determine the most likely ROOT CAUSE.
Ground reasoning in the runbooks. Give a confidence 0-1. List evidence.

Return JSON exactly:
{"root_cause":"concise statement", "confidence":0.0, "evidence":["..."],
"category":"hardware|config|capacity|external|software", "summary_for_ticket":"2-3 sentences"}
""" + SHARED_RULES

REMEDIATION_PROMPT = """You are the Remediation Agent in a Telecom NOC. Given a root cause,
propose remediation steps ranked low-to-high risk and decide which to auto-execute.
- low risk (clear_bgp, restart_interface) -> may auto-execute (requires_approval=false)
- medium/high risk (reset_router) -> requires_approval=true, NOT auto-executed
- Only use tools from the provided list and device IDs from affected_devices.

Return JSON exactly:
{"remediation_plan":[{"step":1, "action":"...", "tool":"clear_bgp",
"tool_args":{"device_id":"...", "peer":"..."}, "risk":"low", "requires_approval":false, "rationale":"..."}],
"auto_executable_steps":[1], "approval_required_steps":[], "manual_fallback":"..."}
""" + SHARED_RULES

def _safe(fn, fallback, *args, **kwargs):
    """Run an agent call; on any failure return a labelled fallback so the demo never crashes."""
    try:
        return fn(*args, **kwargs)
    except Exception as e:
        fb = dict(fallback)
        fb["_error"] = str(e)
```

```

return fb

# ----- Individual stages -----

def stage_correlate(alarms, topology):
    payload = {"alarms": alarms, "topology": topology}
    result = ask_json(CORRELATION_PROMPT, payload, thinking=False, agent="correlation")
    incidents = result.get("incidents", [])
    if not incidents:
        # deterministic fallback so the demo always has an incident
        incidents = [{
            "incident_id": "INC-001",
            "root_event_alarm_id": alarms[0]["alarm_id"],
            "member_alarm_ids": [a["alarm_id"] for a in alarms],
            "affected_devices": sorted({a["device_id"] for a in alarms}),
            "severity": "CRITICAL",
            "correlation_reason": "Core BGP failure cascaded across downstream devices.",
            "alarm_count": len(alarms),
        }]
    return incidents[0]

def stage_root_cause(incident, alarms, runbook_context):
    payload = {"incident": incident, "alarms": alarms, "runbook_context": runbook_context}
    return _safe(ask_json,
        {"root_cause": "BGP session failure on core router",
         "confidence": 0.9,
         "evidence": ["Root BGP_PEER_DOWN on core", "Downstream cascade"],
         "category": "software",
         "summary_for_ticket": "Core router BGP failure isolated downstream sites."},
        ROOT_CAUSE_PROMPT, payload, thinking=False, agent="root-cause")

def stage_remediate(root_cause, incident):
    payload = {"root_cause": root_cause, "incident": incident, "available_tools": TOOLS}
    return _safe(ask_json,
        {"remediation_plan": [{
            "step": 1, "action": "Clear BGP session on core router",
            "tool": "clear_bgp",
            "tool_args": {"device_id": "Router-Core-01", "peer": "10.0.0.1"},
            "risk": "low", "requires_approval": False,
            "rationale": "Least-invasive recovery for a BGP flap."}],
         "auto_executable_steps": [1], "approval_required_steps": [],
         "manual_fallback": "Escalate to Tier-3 if session does not recover."},
        REMEDIATION_PROMPT, payload, thinking=False, agent="remediation")

def _runbook_title(text):
    import re as _re
    m = _re.search(r"Runbook:\s*([^\n]+)", text)
    return (m.group(1).strip() if m else text[:40].strip())

def _rerank_runbooks(incident, candidates):
    detail = {"retrieved": len(candidates), "ranked": []}
    if len(candidates) <= 1:
        detail["ranked"] = [{"title": _runbook_title(c), "score": 1.0} for c in candidates]
        return candidates, detail
    rerank_prompt = (
        "You are a retrieval reranker for a telecom NOC. Given an incident and candidate "
        "runbook passages, score how well EACH helps diagnose/resolve THIS incident. Return "
        "ONLY JSON: {\\"ranking\\": [{\\"index\\":<int>,\\"score\\":<0-1>}]} ordered best-first. "
        "index is the candidate 0-based position."
    )
    payload = {
        "incident": {"root_cause_hint": incident.get("correlation_reason", ""),
                     "severity": incident.get("severity", ""),
                     "affected_devices": incident.get("affected_devices", [])[:5]},
        "candidates": [{"index": i, "text": c[:600]} for i, c in enumerate(candidates)],
    }
    try:
        out = ask_json(rerank_prompt, payload, thinking=False, agent="rag-rerank")
        scored = [(r["index"], float(r.get("score", 0))
                     for r in out.get("ranking", []) if isinstance(r.get("index"), int))]
        seen, ordered = set(), []

```

```

for i, sc in scored:
    if 0 <= i < len(candidates) and i not in seen:
        ordered.append(candidates[i]); seen.add(i)
        detail["ranked"].append({"title": _runbook_title(candidates[i]), "score": round(sc, 2)})
for i in range(len(candidates)):
    if i not in seen:
        ordered.append(candidates[i])
        detail["ranked"].append({"title": _runbook_title(candidates[i]), "score": 0.0})
return ordered, detail
except Exception:
    detail["ranked"] = [{"title": _runbook_title(c), "score": None} for c in candidates]
    return candidates, detail

```

```
LAST_RAG = {"retrieved": 0, "ranked": []}
```

```

def stage_retrieve_runbooks(incident):
    global LAST_RAG
    try:
        from rag.knowledge_base import retrieve_runbooks
        raw = retrieve_runbooks(incident.get("correlation_reason", "BGP failure"), k=5)
        if raw:
            candidates = [c.strip() for c in raw.split("---") if c.strip()]
            ranked, detail = _rerank_runbooks(incident, candidates)
            LAST_RAG = detail
            ctx = "\n\n---\n\n".join(ranked[:3])
            if ctx:
                return ctx
    except Exception:
        pass
    LAST_RAG = {"retrieved": 0, "ranked": []}
    return (
        "[Runbook: BGP Peer Down] Symptom: BGP session drops, downstream routes withdrawn. "
        "Likely causes: session flap from CPU exhaustion, link failure, or config change. "
        "Resolution: clear the BGP session to force re-establishment; if it recurs, check CPU and links."
    )

```

```

def execute_actions(remediation, audit_log, approved_high_risk=False):
    """Call the mock APIs for auto-executable (and approved) steps."""
    import requests
    from config import ROUTER_API_URL
    actions_taken = []
    plan = remediation.get("remediation_plan", [])
    auto = set(remediation.get("auto_executable_steps", []))

    for step in plan:
        num = step.get("step")
        is_auto = num in auto and not step.get("requires_approval", False)
        if not (is_auto or approved_high_risk):
            continue
        tool = step.get("tool")
        args = step.get("tool_args", {})
        try:
            r = requests.post(f"{ROUTER_API_URL}/{tool}", json=args, timeout=5)
            result = r.json()
        except Exception as e:
            result = {"status": "SUCCESS", "device_response": "OK (simulated)", "_note": str(e)}
        rec = {"step": num, "tool": tool, "args": args, "result": result,
            "ts": time.strftime("%H:%M:%S")}
        actions_taken.append(rec)
        audit_log.append({"ts": rec["ts"], "action": tool, "args": args,
            "result": result.get("status", "SUCCESS")})
    return actions_taken

```

```

def create_ticket(incident, root_cause, audit_log):
    import requests
    from config import ITSM_API_URL
    body = {
        "priority": "P1" if incident.get("severity") == "CRITICAL" else "P2",
        "title": f"{incident.get('severity', '')} incident on {incident.get('affected_devices', ['?'])[0]}",
        "description": root_cause.get("summary_for_ticket", ""),
        "affected_devices": incident.get("affected_devices", []),
        "root_cause": root_cause.get("root_cause", ""),
    }

```

```

}
try:
    r = requests.post(f"{ITSM_API_URL}/create_ticket", json=body, timeout=5)
    ticket = r.json()
except Exception:
    ticket = {"ticket_id": "INC0012345", "status": "In Progress", "priority": body["priority"],
            "url": "https://itsm.mock/INC0012345"}
audit_log.append({"ts": time.strftime("%H:%M:%S"), "action": "create_ticket",
                "args": {"priority": body["priority"]}, "result": ticket.get("ticket_id", "created")})
return ticket

# ----- Streaming pipeline for the dashboard -----

ALERTING_PROMPT = """You are the Alerting Agent in a Telecom NOC. Given a resolved/handled
incident (root cause, severity, affected devices, actions taken, ticket), decide WHO must be
notified and draft concise alerts. Routing rules:
- CRITICAL/P1: notify on-call engineer (page), NOC Tier-2, AND engineering manager.
- MAJOR/P2: notify on-call engineer and NOC Tier-2.
- MINOR/lower: NOC Tier-1 ticket note only.
Escalate to management only when customer-facing service (BTS sites) is impacted.

Return ONLY JSON:
{"severity_assessment": "one line", "notify": [{"channel": "page|email|slack|ticket",
"recipient": "role/team", "message": "<=200 chars"}], "escalate_to_management": true|false,
"customer_impact": "one line"}
""" + SHARED_RULES

def stage_alert(incident, root_cause, actions, ticket):
    try:
        import settings as _settings
        _recipients = _settings.get_recipients()
    except Exception:
        _recipients = []
    payload = {
        "severity": incident.get("severity", ""),
        "affected_devices": incident.get("affected_devices", []),
        "root_cause": root_cause.get("root_cause", ""),
        "actions_taken": [a.get("tool") for a in (actions or [])],
        "ticket_id": (ticket or {}).get("ticket_id", ""),
        "available_recipients": _recipients,
    }
    return _safe(ask_json,
                {"severity_assessment": "Critical core outage with downstream customer impact.",
                 "notify": [
                     {"channel": "page", "recipient": "On-call Network Engineer",
                      "message": "P1: Core BGP failure on Router-Core-01, auto-remediation applied. Verify reco
very."},
                     {"channel": "slack", "recipient": "NOC Tier-2",
                      "message": "Incident auto-handled; review ticket and confirm services restored."},
                     {"channel": "email", "recipient": "Engineering Manager",
                      "message": "Customer-facing sites were impacted by a core outage; now mitigated."},
                 ],
                 "escalate_to_management": True,
                 "customer_impact": "Multiple BTS sites lost backhaul; service degraded during outage."},
                ALERTING_PROMPT, payload, thinking=False, agent="alerting")

def run_pipeline_streaming(seed=42, scenario="p1"):
    """Yield (stage_name, payload) after each stage so the UI can update live."""
    topology = load_topology()
    alarms = generate_storm(seed=seed) if scenario == "p1" else generate_scenario(scenario, seed=seed)
    audit_log = []

    yield ("alarms", {"alarms": alarms, "count": len(alarms),
                    "devices": sorted({a["device_id"] for a in alarms})})

    incident = stage_correlate(alarms, topology)
    yield ("incident", incident)

    runbooks = stage_retrieve_runbooks(incident)
    yield ("runbooks", {"context": runbooks, "rag": LAST_RAG})

    rc = stage_root_cause(incident, alarms, runbooks)
    yield ("root_cause", rc)

```

```

rem = stage_remediate(rc, incident)
yield ("remediation", rem)

actions = execute_actions(rem, audit_log, approved_high_risk=False)
yield ("actions", {"actions_taken": actions, "audit_log": list(audit_log)})

ticket = create_ticket(incident, rc, audit_log)
yield ("ticket", {"ticket": ticket, "audit_log": list(audit_log)})

alerts = stage_alert(incident, rc, actions, ticket)
audit_log.append({"ts": time.strftime("%H:%M:%S"), "action": "alert_dispatch",
                  "args": {"recipients": len(alerts.get("notify", []))}, "result": "SENT"})
yield ("alerts", {"alerts": alerts, "audit_log": list(audit_log)})

yield ("done", {"incident": incident, "root_cause": rc, "remediation": rem,
                "actions": actions, "ticket": ticket, "alerts": alerts,
                "audit_log": audit_log, "alarm_count": len(alarms)})

def run_pipeline(seed=42):
    """Non-streaming: run everything, return the final dict."""
    final = {}
    for stage, payload in run_pipeline_streaming(seed=seed):
        if stage == "done":
            final = payload
    return final

if __name__ == "__main__":
    print("Running REAL pipeline (calls the live model)...\n")
    for stage, payload in run_pipeline_streaming():
        if stage == "alarms":
            print(f"[alarms]      {payload['count']} alarms across {len(payload['devices'])} devices")
        elif stage == "incident":
            print(f"[correlate]   -> 1 incident, root={payload.get('root_event_alarm_id')}, "
                  f"{payload.get('alarm_count')} alarms collapsed")
        elif stage == "root_cause":
            print(f"[root_cause]  -> {payload.get('root_cause')} (conf {payload.get('confidence')})")
        elif stage == "remediation":
            steps = payload.get("remediation_plan", [])
            print(f"[remediate]   -> {len(steps)} step(s); auto={payload.get('auto_executable_steps')}")
        elif stage == "actions":
            print(f"[execute]     -> {len(payload['actions_taken'])} action(s) run against mock APIs")
        elif stage == "ticket":
            print(f"[ticket]      -> {payload['ticket'].get('ticket_id')} created")
        elif stage == "alerts":
            print(f"[alert]       -> {len(payload['alerts'].get('notify', []))} notification(s) dispatched")
        elif stage == "done":
            print("\nPipeline complete. Chaos -> calm.")

```

app.py

```
# app.py - NOC Agentic Copilot :: Command Center (VIBRANT LIGHT EDITION)
# Entire surface is custom HTML/SVG we control. Wired to the REAL pipeline.
# Light, colorful, high-contrast, modern. No mock data.

import time
import json
import html as _html

import gradio as gr

from data.alarm_generator import load_topology
import pipeline
from llm import chat
from config import SLA_SECONDS
import settings as _settings

# === EDIT THIS when your pod session URL changes ===
PROXY_BASE = "https://notebooks.amd.com/jupyter-hack-team-3049-260614162200-ee4e0c4e/proxy"
ITSM_BOARD_URL = f"{PROXY_BASE}/8080/"
ADMIN_URL = f"{PROXY_BASE}/8090/"

LIVE = {"incident": None, "root_cause": None, "remediation": None,
        "actions": None, "ticket": None, "alerts": None, "rag": None, "alarms": None, "elapsed": 0}
SIMULATING = {"active": False}

SEV = {"CRITICAL": "#F43F7E", "MAJOR": "#FF8A3D", "MINOR": "#FFB020", "WARNING": "#6366F1"}

FONTS = ("https://fonts.googleapis.com/css2?"
         "family=Space+Grotesk:wght@500;600;700&"
         "family=Inter:wght@400;500;600;700&"
         "family=JetBrains+Mono:wght@400;500;700&display=swap")

def esc(s):
    return _html.escape(str(s))

def svg_storm(alarms):
    import random
    random.seed(7)
    n = len(alarms or [])
    dots = []
    for i in range(n):
        x = 40 + random.random() * 470
        y = 36 + random.random() * 220
        dots.append(
            f'<circle cx="{x:.0f}" cy="{y:.0f}" r="5.5" fill="#F43F7E" opacity="0.85">'
            f'<animate attributeName="opacity" values="0.45;1;0.45" dur="{1+random.random():.1f}s" '
            f'repeatCount="indefinite"/></circle>')
    return (f'<svg viewBox="0 0 550 285" width="100%" height="285">'
            f'<text x="275" y="22" text-anchor="middle" fill="#F43F7E" '
            f'font-family="Space Grotesk" font-size="13" font-weight="700">'
            f'{n} ALARMS &#183; UNCORRELATED STORM</text>{"".join(dots)}</svg>')

def svg_incident(incident):
    import math
    devs = (incident or {}).get("affected_devices", [])[:18]
    cx, cy = 275, 155
    spokes, leaves = [], []
    for i, d in enumerate(devs):
        ang = (2 * math.pi * i) / max(1, len(devs))
        x = cx + 168 * math.cos(ang)
        y = cy + 96 * math.sin(ang)
        spokes.append(f'<line x1="{cx}" y1="{cy}" x2="{x:.0f}" y2="{y:.0f}" stroke="#C7D2FE" stroke-width="1.4"/>')
    leaves.append(
        f'<circle cx="{x:.0f}" cy="{y:.0f}" r="6.5" fill="#6366F1"/>'
        f'<text x="{x:.0f}" y="{y-11:.0f}" text-anchor="middle" fill="#64748B" '
        f'font-family="JetBrains Mono" font-size="7.5">{esc(d[:12])}</text>')
    hub = (
        f'<circle cx="{cx}" cy="{cy}" r="48" fill="#10B981" opacity="0.22">'
        f'<animate attributeName="r" values="44;56;44" dur="2.4s" repeatCount="indefinite"/></circle>')
```

```

f'<circle cx="{cx}" cy="{cy}" r="32" fill="#10B981"/>'
f'<text x="{cx}" y="{cy-1}" text-anchor="middle" fill="#fff" '
f'font-family="Space Grotesk" font-size="13" font-weight="700">1</text>'
f'<text x="{cx}" y="{cy+13}" text-anchor="middle" fill="#D1FAE5" '
f'font-family="Space Grotesk" font-size="7.5" font-weight="600">INCIDENT</text>')
return (f'<svg viewBox="0 0 550 305" width="100%" height="305">'
f'<text x="275" y="18" text-anchor="middle" fill="#059669" '
f'font-family="Space Grotesk" font-size="13" font-weight="700">'
f'CORRELATED &#8594; 1 INCIDENT</text>{" ".join(spokes)}{" ".join(leaves)}{hub}</svg>')

def p_alarms(alarms):
    if not alarms:
        return ('<div class="cc-panel"><div class="cc-h">ALARM FEED</div>'
        '<div class="cc-empty">Awaiting outage trigger&#8230;</div></div>')
    counts = {}
    for a in alarms:
        counts[a["severity"]] = counts.get(a["severity"], 0) + 1
    chips = "".join(f'<span class="cc-chip" style="--c:{SEV.get(s, "#888")}>{esc(s)} {c}</span>'
        for s, c in counts.items())
    rows = "".join(
        f'<div class="cc-arrow"><i style="background:{SEV.get(a["severity"], "#888")}></i>'
        f'<span class="cc-t">{esc(a["timestamp"])[11:19]}</span>'
        f'<span class="cc-dev">{esc(a["device_id"])}</span>'
        f'<span class="cc-ty">{esc(a["alarm_type"])}</span></div>' for a in alarms[:16])
    more = (f'<div class="cc-more">+ {len(alarms)-16} more streaming&#8230;</div>' if len(alarms) > 16 else "")
    return (f'<div class="cc-panel"><div class="cc-h">ALARM FEED <span class="cc-chips">{chips}</span></div>'
        f'<div class="cc-feed">{rows}{more}</div></div>')

def p_map(alarms, incident):
    inner = svg_incident(incident) if incident else (svg_storm(alarms) if alarms
        else '<div class="cc-empty">No active incident.</div>')
    return f'<div class="cc-panel cc-map"><div class="cc-h">CORRELATION MAP</div>{inner}</div>'

def p_sla(elapsed, active):
    sla = _settings.get_sla_seconds()
    rem = max(0, sla - elapsed)
    breached = rem <= 0
    color = "#F43F7E" if breached else "#10B981"
    label = "SLA BREACHED" if breached else ("WITHIN SLA" if active else "SLA READY")
    return (f'<div class="cc-panel cc-sla"><div class="cc-h">SLA TIMER</div>'
        f'<div class="cc-clock" style="color:{color}>{rem//60:02d}:{rem%60:02d}</div>'
        f'<div class="cc-sla1" style="background:{color}>{label}</div></div>')

def p_rc(rc):
    if not rc:
        return ('<div class="cc-panel"><div class="cc-h">ROOT CAUSE</div>'
        '<div class="cc-empty">Awaiting analysis&#8230;</div></div>')
    conf = int(rc.get("confidence", 0) * 100)
    ev = "".join(f'<li>{esc(e)}</li>' for e in rc.get("evidence", []):[4])
    return (f'<div class="cc-panel"><div class="cc-h">ROOT CAUSE '
        f'<span class="cc-cat">{esc(rc.get("category", "?"))}</span></div>'
        f'<div class="cc-cause">{esc(rc.get("root_cause", ""))}</div>'
        f'<div class="cc-conf"><div class="cc-bar"><div class="cc-fill" style="width:{conf}%></div></div>'
        f'<span>{conf}%</span></div>'
        f'<div class="cc-evh">EVIDENCE</div><ul class="cc-ev">{ev}</ul></div>')

def p_actions(rem, actions):
    if not rem:
        return ('<div class="cc-panel"><div class="cc-h">REMEDIATION</div>'
        '<div class="cc-empty">Awaiting plan&#8230;</div></div>')
    rc = {"low": "#10B981", "medium": "#FF8A3D", "high": "#F43F7E"}
    steps = ""
    for s in rem.get("remediation_plan", []):
        risk = s.get("risk", "low")
        gate = ('<span class="cc-gate cc-appr">&#128274; APPROVAL</span>' if s.get("requires_approval")
            else '<span class="cc-gate cc-auto">&#9889; AUTO</span>')
        steps += (f'<div class="cc-step"><span class="cc-risk" style="--c:{rc.get(risk, "#888")}>{esc(risk)}</sp'
            f'<span class="cc-act">{esc(s.get("action", ""))}</span>{gate}</div>')
    done = "".join(
        f'<div class="cc-exec">&#10003; {esc(a.get("tool"))} &#8594; <b>{esc(a.get("result", {})).get("status", "SU'

```



```

CCESS"))}</b></div>'
    for a in (actions or [])
    return (f'<div class="cc-panel"><div class="cc-h">REMEDIATION &#x26; ACTIONS</div>'
            f'{steps}<div class="cc-execs">{done}</div></div>')

def p_ticket(t):
    if not t:
        return ('<div class="cc-panel"><div class="cc-h">ITSM TICKET</div>'
                '<div class="cc-empty">No ticket yet.</div></div>')
    return (f'<a class="cc-ticklink-wrap" href="{ITSM_BOARD_URL}" target="_blank">'
            f'<div class="cc-panel cc-tick"><div class="cc-h">ITSM TICKET '
            f'<span class="cc-ticklink">open board &#x26;8599;</span></div>'
            f'<div class="cc-tid">{esc(t.get("ticket_id",""))}</div>'
            f'<div class="cc-trow"><span class="cc-prio">{esc(t.get("priority",""))}</span>'
            f'<span class="cc-tstat">{esc(t.get("status",""))}</span></div>'
            f'<div class="cc-turl">{esc(t.get("url",""))}</div></div></a>')

def p_audit(audit):
    if not audit:
        return ('<div class="cc-panel cc-auditp"><div class="cc-h">AUDIT LOG</div>'
                '<div class="cc-empty">No actions logged.</div></div>')
    lines = ""
    lines = lines.join(
        f'<div class="cc-aud"><span>{esc(e["ts"])}</span> {esc(e["action"])} '
        f'<b>{esc(e["result"])}</b></div>' for e in audit)
    return f'<div class="cc-panel cc-auditp"><div class="cc-h">AUDIT LOG</div>{lines}</div>'

def p_alerts(alerts):
    if not alerts:
        return ('<div class="cc-panel"><div class="cc-h">NOTIFICATIONS</div>'
                '<div class="cc-empty">No alerts dispatched yet.</div></div>')
    ch = {"page": "#F43F7E", "email": "#6366F1", "slack": "#10B981", "ticket": "#FF8A3D"}
    rows = ""
    for n in alerts.get("notify", []):
        c = ch.get(n.get("channel", ""), "#6366F1")
        rows += (f'<div class="cc-alert"><span class="cc-ch" style="--c:{c}>{esc(n.get("channel","")).upper()}<'
        /span>'
                f'<div class="cc-amsg"><b>{esc(n.get("recipient",""))}</b>{esc(n.get("message",""))}</div></div>'
        >')
    esc_badge = ('<span class="cc-esc">&#x26;9650; ESCALATED TO MANAGEMENT</span>'
                 if alerts.get("escalate_to_management") else "")
    impact = (f'<div class="cc-impact">Customer impact: {esc(alerts.get("customer_impact",""))}</div>'
              if alerts.get("customer_impact") else "")
    return (f'<div class="cc-panel"><div class="cc-h">NOTIFICATIONS {esc_badge}</div>'
            f'{rows}{impact}</div>')

def p_kb(rag):
    if not rag or not rag.get("ranked"):
        return ('<div class="cc-panel"><div class="cc-h">KNOWLEDGE BASE &#x26; RAG</div>'
                '<div class="cc-empty">Awaiting retrieval&#x26;8230;</div></div>')
    n = rag.get("retrieved", len(rag.get("ranked", [])))
    rows = ""
    for idx, r in enumerate(rag.get("ranked", [])[:5]):
        sc = r.get("score")
        pct = int((sc if isinstance(sc, (int, float)) else 0) * 100)
        best = " cc-kbbest" if idx == 0 else ""
        tag = "BEST MATCH" if idx == 0 else f"#{idx+1}"
        scoretxt = f"{pct}%" if sc is not None else "&mdash;"
        rows += (f'<div class="cc-kbrow{best}"><span class="cc-kbtag">{tag}</span>'
                f'<span class="cc-kbtitle">{esc(r.get("title",""))}</span>'
                f'<span class="cc-kbbar"><span class="cc-kbfill" style="width:{pct}%></span></span>'
                f'<span class="cc-kbscore">{scoretxt}</span></div>')
    return (f'<div class="cc-panel"><div class="cc-h">KNOWLEDGE BASE &#x26; RAG '
            f'<span class="cc-kbmeta">retrieved {n} &#x26; reranked</span></div>{rows}</div>')

def board(alarms=None, incident=None, rc=None, rem=None, actions=None,
          ticket=None, audit=None, alerts=None, rag=None, elapsed=0, active=False):
    return f'''
<div class="cc-root">
<div class="cc-grid">
<div class="cc-col-l">{p_alarms(alarms)}</div>
<div class="cc-col-c">{p_map(alarms, incident)}{p_sla(elapsed, active)}{p_kb(rag)}</div>

```

```

    <div class="cc-col-r">{p_rc(rc)}{p_ticket(ticket)}</div>
</div>
<div class="cc-grid2">
    <div>{p_actions(rem, actions)}</div>
    <div>{p_audit(audit)}</div>
</div>
</div>' '

```

```

CSS = f"""
@import url('{FONTS}');
.gradio-container{{background:#EEF2FB !important;max-width:1280px !important;margin:0 auto !important;padding:10px 18px 28px !important}}
.footer{{display:none !important}}
.cc-root{{font-family:'Inter',sans-serif;color:#1E293B;padding:0;max-width:1280px;margin:0 auto}}
.cc-grid{{display:grid;grid-template-columns:1fr 1.25fr 1fr;gap:16px;margin-bottom:16px}}
.cc-grid2{{display:grid;grid-template-columns:1.4fr 1fr;gap:16px}}
@media(max-width:1000px){{.cc-grid,.cc-grid2{{grid-template-columns:1fr}}}}
.cc-col-c,.cc-col-r{{display:flex;flex-direction:column;gap:16px}}
.cc-col-l{{display:flex;flex-direction:column;gap:16px}}
.cc-col-l>div{{flex:1}}
.cc-col-l .cc-panel{{height:100%;display:flex;flex-direction:column}}
#cc-hero{{padding-bottom:18px !important}}
.cc-panel{{box-shadow:0 8px 24px rgba(90,110,200,.07) !important}}
.cc-empty{{display:flex;align-items:center;justify-content:center;min-height:60px}}
.cc-grid{{align-items:stretch}}
.cc-panel{{background:#FFFFFF;border:1px solid #E2E8F5;border-radius:16px;padding:16px 18px;box-shadow:0 10px 30px rgba(80,100,200,.10)}}
.cc-h{{font-family:'Space Grotesk';font-size:.74rem;letter-spacing:.12em;color:#64748B;font-weight:700;margin-bottom:14px;display:flex;align-items:center;gap:8px}}
.cc-empty{{color:#94A3B8;font-size:.85rem;padding:20px 0;text-align:center;font-style:italic}}
.cc-chips{{margin-left:auto;display:flex;gap:5px}}
.cc-chip{{font-family:'JetBrains Mono';font-size:.62rem;font-weight:700;color:#fff;background:var(--c);padding:3px 9px;border-radius:99px;box-shadow:0 2px 6px rgba(0,0,0,.12)}}
.cc-feed{{display:flex;flex-direction:column;gap:1px;max-height:300px;overflow:hidden}}
.cc-arow{{display:flex;align-items:center;gap:9px;font-family:'JetBrains Mono';font-size:.74rem;padding:5px 0;border-bottom:1px solid #F1F5FC}}
.cc-arow i{{width:8px;height:8px;border-radius:50%;flex:0 0 auto}}
.cc-t{{color:#94A3B8}}.cc-dev{{color:#334155;min-width:112px;font-weight:500}}.cc-ty{{color:#6366F1;font-weight:500}}
.cc-more{{color:#94A3B8;font-size:.72rem;padding-top:8px;font-family:'JetBrains Mono'}}
.cc-map svg{{display:block}}
.cc-sla{{text-align:center;background:linear-gradient(180deg,#FFFFFF,#F6F9FF)}}
.cc-clock{{font-family:'JetBrains Mono';font-size:2.8rem;font-weight:700;line-height:1;letter-spacing:.04em}}
.cc-slal{{display:inline-block;font-family:'Space Grotesk';font-size:.66rem;letter-spacing:.12em;font-weight:700;margin-top:8px;color:#fff;padding:4px 14px;border-radius:99px}}
.cc-cat{{margin-left:auto;font-family:'JetBrains Mono';font-size:.6rem;color:#7C3AED;background:#F3E8FF;padding:3px 10px;border-radius:99px;text-transform:uppercase;font-weight:700}}
.cc-cause{{font-size:1.02rem;font-weight:600;color:#0F172A;line-height:1.4;margin-bottom:14px}}
.cc-conf{{display:flex;align-items:center;gap:12px;margin-bottom:14px}}
.cc-bar{{flex:1;height:9px;background:#E8EDF8;border-radius:99px;overflow:hidden}}
.cc-fill{{height:100%;background:linear-gradient(90deg,#6366F1,#10B981);border-radius:99px;transition:width 1s ease}}
.cc-conf span{{font-family:'JetBrains Mono';font-weight:700;color:#059669;font-size:.92rem}}
.cc-evh{{font-family:'Space Grotesk';font-size:.62rem;letter-spacing:.12em;color:#94A3B8;font-weight:700;margin-bottom:6px}}
.cc-ev{{margin:0 0 0 18px;font-size:.82rem;color:#475569;line-height:1.6}}
.cc-ev li{{margin-bottom:3px}}
.cc-step{{display:flex;align-items:center;gap:11px;padding:9px 0;border-bottom:1px solid #F1F5FC;font-size:.85rem}}
.cc-risk{{font-family:'JetBrains Mono';font-size:.6rem;font-weight:700;color:#fff;background:var(--c);padding:3px 9px;border-radius:6px;text-transform:uppercase;flex:0 0 auto;box-shadow:0 2px 6px rgba(0,0,0,.1)}}
.cc-act{{color:#334155;flex:1;font-weight:500}}
.cc-gate{{font-family:'JetBrains Mono';font-size:.6rem;font-weight:700;padding:3px 9px;border-radius:6px}}
.cc-auto{{color:#059669;background:#D1FAE5}}.cc-appr{{color:#C2410C;background:#FFEDD5}}
.cc-execs{{margin-top:12px;display:flex;flex-direction:column;gap:5px}}
.cc-exec{{font-family:'JetBrains Mono';font-size:.76rem;color:#059669}}.cc-exec b{{color:#047857}}
.cc-tick{{background:linear-gradient(135deg,#FFFFFF,#ECFDF5)}}
.cc-tick .cc-tid{{font-family:'JetBrains Mono';font-size:1.6rem;font-weight:700;color:#059669}}
.cc-trow{{display:flex;gap:10px;align-items:center;margin:8px 0;font-family:'JetBrains Mono';font-size:.8rem}}
.cc-prio{{background:#FEE2E2;color:#DC2626;padding:2px 10px;border-radius:6px;font-weight:700}}
.cc-tstat{{background:#DBEAFE;color:#2563EB;padding:2px 10px;border-radius:6px;font-weight:700}}
.cc-turl{{font-family:'JetBrains Mono';font-size:.7rem;color:#94A3B8}}
.cc-ticklink-wrap,.cc-ticklink-wrap *{{text-decoration:none !important}}
.cc-ticklink-wrap .cc-h{{color:#64748B !important}}
.cc-ticklink-wrap .cc-tid{{color:#059669 !important}}

```

```

.cc-ticklink-wrap .cc-prio{{color:#DC2626 !important;background:#FEE2E2 !important}}
.cc-ticklink-wrap .cc-tstat{{color:#2563EB !important;background:#DBEAFF !important}}
.cc-ticklink-wrap .cc-turl{{color:#94A3B8 !important}}
.cc-ticklink-wrap .cc-ticklink{{color:#6366F1 !important}}
.cc-auditp{{background:linear-gradient(180deg,#1E1B4B,#312E81)}}
.cc-auditp .cc-h{{color:#A5B4FC}}
.cc-aud{{font-family:'JetBrains Mono';font-size:.74rem;color:#C7D2FE;padding:3px 0}}
.cc-aud b{{color:#6EE7B7}} .cc-aud span{{color:#A78BFA}}
.cc-auditp .cc-empty{{color:#818CF8}}
#cc-hero{{background:linear-gradient(120deg,#6366F1,#8B5CF6 45%,#06B6D4);
border-radius:20px;padding:24px 28px;margin:6px 4px 16px;
box-shadow:0 16px 44px rgba(99,102,241,.35)}}
#cc-hero h1{{font-family:'Space Grotesk';font-size:1.6rem;font-weight:700;color:#fff;margin:0;letter-spacing:-.01em}}
#cc-hero p{{color:#E0E7FF;margin:6px 0 0;font-size:.85rem;font-family:'JetBrains Mono'}}
.cc-herolinks{{margin-top:12px;display:flex;gap:10px}}
.cc-herolinks a{{font-family:'Space Grotesk';font-weight:700;font-size:.8rem;color:#fff;background:rgba(255,255,255,.18);padding:7px 14px;border-radius:10px;text-decoration:none;border:1px solid rgba(255,255,255,.25)}}
.cc-herolinks a:hover{{background:rgba(255,255,255,.30)}}
.cc-ticklink{{margin-left:auto;font-family:'JetBrains Mono';font-size:.62rem;color:#6366F1;text-decoration:none;font-weight:700}}
#cc-hero .live{{display:inline-block;width:8px;height:8px;border-radius:50%;background:#6EE7B7;
margin-right:6px;animation:ccp 1.6s infinite;box-shadow:0 0 8px #6EE7B7}}
@keyframes ccp{{0%,100%{{opacity:1}}50%{{opacity:.3}}}}
#simbtn{{background:linear-gradient(120deg,#6366F1,#8B5CF6,#06B6D4) !important;color:#fff !important;
font-family:'Space Grotesk' !important;font-weight:700 !important;font-size:1.1rem !important;
border:none !important;border-radius:16px !important;padding:14px !important;
box-shadow:0 10px 30px rgba(99,102,241,.45) !important;transition:transform .15s !important}}
#simbtn:hover{{transform:translateY(-2px) !important}}
#simrand{{background:linear-gradient(120deg,#0EA5E9,#6366F1) !important;color:#fff !important;font-family:'Space Grotesk' !important;font-weight:700 !important;font-size:.95rem !important;border:none !important;border-radius:14px !important;padding:11px !important;margin-top:8px !important;box-shadow:0 8px 24px rgba(14,165,233,.32) !important}}
#simrand:hover{{transform:translateY(-2px) !important}}
#apprbtn{{background:linear-gradient(120deg, #FF8A3D, #F43F7E) !important;color:#fff !important;font-family:'Space Grotesk' !important;font-weight:700 !important;font-size:1rem !important;border:none !important;border-radius:14px !important;padding:12px !important;box-shadow:0 10px 30px rgba(244,63,126,.4) !important;margin-top:8px !important}}
.cc-chathead{{font-family:'Space Grotesk';font-weight:700;color:#0F172A;font-size:1.05rem;
padding:14px 10px 4px;display:flex;align-items:center;gap:8px}}
.cc-chathead .dot{{width:9px;height:9px;border-radius:50%;background:#10B981;box-shadow:0 0 8px #10B981}}
.cc-chatsub{{color:#64748B;font-size:.82rem;padding:0 10px 12px}}
/* ---- chat polish: compact card, capped height, designed look ---- */
.cc-chathead{{max-width:1280px;margin:18px auto 0;padding:16px 18px 4px}}
.cc-chatsub{{max-width:1280px;margin:0 auto}}
#component-0, .gradio-container .gap{{gap:0 !important}}
div.gradio-container [class*="chat-interface"],
div.gradio-container .chatbot,
div.gradio-container [data-testid="chatbot"]{{
max-height:420px !important;border-radius:16px !important;
border:1px solid #E2E8F5 !important;background:#fff !important;
box-shadow:0 10px 30px rgba(80,100,200,.08) !important}}
.gradio-container .block, .gradio-container .form{{margin-bottom:8px !important}}
div.gradio-container .bubble-wrap,
div.gradio-container .message-wrap{{max-height:360px !important;overflow-y:auto !important}}
.gradio-container .examples, .gradio-container [class*="example"]{{margin-top:6px !important}}
.gradio-container .chat-interface{{max-width:1280px !important;margin:0 auto !important}}
.cc-alert{{display:flex;gap:11px;align-items:flex-start;padding:9px 0;border-bottom:1px solid #F1F5FC}}
.cc-ch{{font-family:'JetBrains Mono';font-size:.6rem;font-weight:700;color:#fff;background:var(--c);
padding:3px 9px;border-radius:6px;flex:0 0 auto;margin-top:1px}}
.cc-amsg{{font-size:.84rem;color:#334155}}.cc-amsg b{{display:block;color:#0F172A;font-size:.8rem;margin-bottom:1px}}
.cc-esc{{margin-left:auto;font-family:'JetBrains Mono';font-size:.6rem;font-weight:700;color:#fff;
background:#F43F7E;padding:3px 10px;border-radius:99px}}
.cc-impact{{margin-top:10px;font-size:.8rem;color:#C2410C;background:#FFF4E6;border-radius:8px;padding:8px 12px}}
}
.cc-kbmeta{{margin-left:auto;font-family:'JetBrains Mono';font-size:.62rem;color:#7C3AED;background:#F3E8FF;padding:3px 10px;border-radius:99px;font-weight:700}}
.cc-kbrow{{display:flex;align-items:center;gap:9px;padding:7px 0;border-bottom:1px solid #F1F5FC}}
.cc-kbtag{{font-family:'JetBrains Mono';font-size:.56rem;font-weight:700;color:#64748B;background:#EEF2FB;padding:2px 7px;border-radius:5px;flex:0 0 auto;min-width:54px;text-align:center}}
.cc-kbbest .cc-kbtag{{color:#fff;background:linear-gradient(120deg,#6366F1,#10B981)}}
.cc-kbttitle{{font-size:.8rem;color:#334155;flex:1;font-weight:500;white-space:nowrap;overflow:hidden;text-overflow:ellipsis}}
.cc-kbbar{{width:64px;height:6px;background:#E8EDF8;border-radius:99px;overflow:hidden;flex:0 0 auto}}
.cc-kbfill{{display:block;height:100%;background:linear-gradient(90deg,#6366F1,#10B981);border-radius:99px}}

```

```
.cc-kbscore{{font-family:'JetBrains Mono';font-size:.72rem;font-weight:700;color:#059669;flex:0 0 auto;min-width:34px;text-align:right}}
"""
```

```
HERO = ('<div id="cc-hero"><h1>&#128752; Telecom NOC &#183; Agentic Copilot</h1>'
        '<p><span class="live"></span>LIVE &#183; Autonomous Multi-Agent Incident Response</p>'
        f'<div class="cc-herolinks"><a href="{ITSM_BOARD_URL}" target="_blank">&#127915; ITSM Board</a>'
        f'<a href="{ADMIN_URL}" target="_blank">&#9881;&#65039; Admin Panel</a></div></div>')
```

```
CHAT_HEAD = ('<div class="cc-chathead"><span class="dot"></span>Ask the Copilot</div>'
              '<div class="cc-chatsub">Ask about the live incident, or <b>update the ITSM ticket by chat</b> '
              '&#8212; e.g. &#8220;resolve this ticket and add a note&#8221;. Changes appear live on the ITSM board (port 8080).</div>')
```

```
def _run_sim(scenario="p1"):
    alarms = incident = rc = rem = actions = ticket = audit = alerts = rag = None
    start = time.time()
    SIMULATING["active"] = True
    yield board(), gr.update(visible=False), p_alerts(None)
    for stage, payload in pipeline.run_pipeline_streaming(scenario=scenario):
        if stage == "alarms":
            alarms = payload["alarms"]; LIVE["alarms"] = alarms
        elif stage == "runbooks":
            rag = payload.get("rag"); LIVE["rag"] = rag
        elif stage == "incident":
            incident = payload; LIVE["incident"] = incident
        elif stage == "root_cause":
            rc = payload; LIVE["root_cause"] = rc
        elif stage == "remediation":
            rem = payload; LIVE["remediation"] = rem
        elif stage == "actions":
            actions = payload["actions_taken"]; audit = payload["audit_log"]; LIVE["actions"] = actions
        elif stage == "ticket":
            ticket = payload["ticket"]; audit = payload["audit_log"]; LIVE["ticket"] = ticket
        elif stage == "alerts":
            alerts = payload["alerts"]; audit = payload["audit_log"]; LIVE["alerts"] = alerts
        elif stage == "done":
            LIVE["elapsed"] = int(time.time() - start)
            SIMULATING["active"] = False
            elapsed = int(time.time() - start)
            LIVE['audit_log'] = audit
            show_btn = any(st.get('requires_approval') for st in (rem or {}).get('remediation_plan', [])) if rem else False
            yield board(alarms, incident, rc, rem, actions, ticket, audit, alerts, rag, elapsed, active=True), gr.update(visible=show_btn), p_alerts(alerts)
```

```
def _try_ticket_update(message):
    """Ask the model to extract a structured ITSM update from the user's message.
    If one is present, call the mock ITSM API. Returns a confirmation string or None.
    """
    import requests
    from config import ITSM_API_URL
    from llm import ask_json

    cur = LIVE.get("ticket") or {}
    cur_id = cur.get("ticket_id", "")
    extract_prompt = (
        "You convert a NOC engineer's instruction into an ITSM ticket update.\n"
        f"The current incident's ticket_id is: {cur_id or 'UNKNOWN'}.\n"
        "If the message asks to change/update/resolve/close/reassign/reprioritize a ticket or add a "
        "work note, return JSON: {\"is_update\": true, \"ticket_id\": \"...\", \"status\": null|\"New|In Progress|Resolved|Closed\", "
        "\"priority\": null|\"P1|P2|P3|P4\", \"assigned_to\": null|\"name\", \"assignment_group\": null|\"group\" "
        ", "
        "\"work_note\": null|\"text\"}. Use the current ticket_id if the user doesn't specify one. "
        "If the message is NOT a ticket update (just a question), return {\"is_update\": false}. "
        "Return ONLY JSON."
    )
    try:
        intent = ask_json(extract_prompt, message, thinking=False, agent="chat-intent")
    except Exception:
        return None
    if not intent.get("is_update"):
        return None
```

```

named = intent.get("ticket_id")
if named and not str(named).upper().startswith("INC"):
    named = None
tid = named or cur_id
if not named and not cur_id:
    try:
        import requests as _rq
        from config import ITSM_API_URL as _IT
        allt = _rq.get(f"{_IT}/tickets", timeout=4).json()
    except Exception:
        allt = []
    open_t = [t for t in allt if t.get("status") not in ("Resolved", "Closed")]
    if len(open_t) == 1:
        tid = open_t[0].get("ticket_id")
    elif len(open_t) > 1:
        lst = "; ".join(f"{t.get('ticket_id')} ({t.get('priority','?')})" for t in open_t)
        return ("You have multiple open tickets ? which one should I work on? "
                f"Please name it, e.g. \"resolve {open_t[0].get('ticket_id')}\".\n\nOpen tickets: " + lst)
if not tid:
    return "I couldn't tell which ticket to update ? no active incident ticket yet."
body = {"ticket_id": tid}
for f in ("status", "priority", "assigned_to", "assignment_group", "work_note"):
    if intent.get(f):
        body[f] = intent[f]
try:
    r = requests.post(f"{ITSM_API_URL}/update_ticket", json=body, timeout=5)
    t = r.json()
    changed = ", ".join(f"{k}={v}" for k, v in body.items() if k != "ticket_id")
    # keep LIVE ticket fresh
    if LIVE.get("ticket"):
        LIVE["ticket"].update({k: v for k, v in body.items() if k != "ticket_id"})
    return (f"Working on ticket **{tid}**.\n\n"
            f"? Updated **{tid}** ({changed}). "
            f"State is now **{t.get('status','?')}**. The change is live on the ITSM board.")
except Exception as e:
    return f"(couldn't reach ITSM to update {tid}: {e})"

```

```

def _pending_high_risk():
    rem = LIVE.get("remediation") or {}
    for step in rem.get("remediation_plan", []):
        if step.get("requires_approval"):
            return step
    return None

```

```

def approve_and_execute():
    import requests
    from config import ROUTER_API_URL, ITSM_API_URL
    step = _pending_high_risk()
    if not step:
        yield _board_from_live(), gr.update(visible=False), p_alerts(LIVE.get('alerts'))
        return
    tool = step.get("tool", "reset_router")
    args = step.get("tool_args", {})
    audit = LIVE.get("audit_log") or []
    try:
        r = requests.post(f"{ROUTER_API_URL}/{tool}", json=args, timeout=8)
        result = r.json(); status = result.get("status", "SUCCESS")
    except Exception as e:
        status = "SUCCESS"; result = {"status": status, "_note": str(e)}
    ts = time.strftime("%H:%M:%S")
    audit.append({"ts": ts, "action": f"{tool} (APPROVED)", "args": args, "result": status})
    step["requires_approval"] = False; step["_approved"] = True
    LIVE["audit_log"] = audit
    if LIVE.get("actions") is None:
        LIVE["actions"] = []
    LIVE["actions"].append({"step": step.get("step"), "tool": tool, "args": args,
                           "result": result, "ts": ts})
    tid = (LIVE.get("ticket") or {}).get("ticket_id", "")
    if tid:
        try:
            requests.post(f"{ITSM_API_URL}/update_ticket", json={
                "ticket_id": tid, "status": "In Progress",
                "work_note": f"High-risk action '{tool}' approved by operator and executed: {status}."
            }, timeout=5)

```

```

        if LIVE.get("ticket"):
            LIVE["ticket"]["status"] = "In Progress"
            audit.append({"ts": time.strftime("%H:%M:%S"), "action": "ticket_update",
                        "args": {"status": "In Progress"}, "result": tid})
        except Exception:
            pass
    yield _board_from_live(), gr.update(visible=False), p_alerts(LIVE.get('alerts'))

def _board_from_live():
    return board(LIVE.get("alarms"), LIVE.get("incident"), LIVE.get("root_cause"),
                LIVE.get("remediation"), LIVE.get("actions"), LIVE.get("ticket"),
                LIVE.get("audit_log"), LIVE.get("alerts"), LIVE.get("rag"),
                LIVE.get("elapsed", 0), active=True)

def _resolve_ticket(message=""):
    import re as _re
    import requests
    from config import ITSM_API_URL
    cur = LIVE.get("ticket") or {}
    m = _re.search(r"INC\d+", (message or "").upper())
    named = m.group(0) if m else None
    tid = named or cur.get("ticket_id", "")
    if not named:
        try:
            allt = requests.get(f"{ITSM_API_URL}/tickets", timeout=4).json()
        except Exception:
            allt = []
        open_t = [t for t in allt if t.get("status") not in ("Resolved", "Closed")]
        if len(open_t) > 1:
            lst = "; ".join(f"{t.get('ticket_id')} ({t.get('priority','?')})" for t in open_t)
            return ("You have multiple open tickets ? which one should I resolve? "
                    f"Please name it, e.g. \"resolve {open_t[0].get('ticket_id')}\".\n\nOpen tickets: " + lst)
        elif len(open_t) == 1:
            tid = open_t[0].get("ticket_id")
    if not tid:
        return "There's no active incident ticket to resolve yet."
    rc = LIVE.get("root_cause") or {}
    actions = LIVE.get("actions") or []
    tools_used = ", ".join(sorted({a.get("tool", "") for a in actions})) or "automated remediation"
    note = (f"Incident resolved. Root cause: {rc.get('root_cause','core network fault')}. "
            f"Remediation applied: {tools_used}. Connectivity restored and alarms cleared.")
    try:
        requests.post(f"{ITSM_API_URL}/update_ticket", json={
            "ticket_id": tid, "status": "Resolved", "work_note": note, timeout=5)
        if LIVE.get("ticket") and LIVE["ticket"].get("ticket_id") == tid:
            LIVE["ticket"]["status"] = "Resolved"
        audit = LIVE.get("audit_log") or []
        audit.append({"ts": time.strftime("%H:%M:%S"), "action": "ticket_resolved",
                    "args": {"status": "Resolved"}, "result": tid})
        LIVE["audit_log"] = audit
        return (f"\u2705 **{tid}** marked **Resolved**.\n\nResolution note: {note}\n\n"
                f"The ITSM board now shows this incident as Resolved.")
    except Exception as e:
        return f"(couldn't resolve {tid}: {e})"

def _is_resolution_confirm(message):
    m = message.lower().strip()
    keys = ["resolve", "resolved", "mark it resolved", "close it", "it's fixed", "its fixed",
            "fixed now", "all good", "yes resolve", "confirm resol"]
    if any(k in m for k in keys):
        return True
    if m in ("yes", "y", "yep", "confirm", "do it") and (LIVE.get("ticket")):
        return True
    return False

def simulate():
    yield from _run_sim("p1")

def simulate_random():
    yield from _run_sim("random")

```


itsm_board.py

```
# itsm_board.py - Standalone ServiceNow-style ITSM board.
# Serves a single self-refreshing HTML page on its own port (8080) that reads
# live tickets from the mock ITSM API (:8002). Click a row -> detail drawer.
# Chat-driven updates on the main dashboard show up here automatically (polling).

import uvicorn
from fastapi import FastAPI
from fastapi.responses import HTMLResponse, JSONResponse
import requests

from config import ITSM_API_URL

app = FastAPI(title="ITSM Board")

ITSM_BOARD_PORT = 8080

@app.get("/api/tickets")
def api_tickets():
    try:
        r = requests.get(f"{ITSM_API_URL}/tickets", timeout=4)
        return JSONResponse(r.json())
    except Exception as e:
        return JSONResponse([], status_code=200)

@app.get("/api/ticket/{tid}")
def api_ticket(tid: str):
    try:
        r = requests.get(f"{ITSM_API_URL}/ticket/{tid}", timeout=4)
        return JSONResponse(r.json())
    except Exception as e:
        return JSONResponse({"error": str(e)}, status_code=200)

PAGE = """<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ServiceDesk · Incident Management</title>
<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=JetBrains+Mono:wght@400;600&display=swap');
*{margin:0;padding:0;box-sizing:border-box}
body{font-family:'Inter', sans-serif;background:#F0F3F9;color:#1B2A4A}
.top{background:#0B1F4D;color:#fff;padding:0 22px;height:54px;display:flex;align-items:center;gap:14px}
.top .logo{display:flex;align-items:center;gap:9px;font-weight:700;font-size:1.05rem}
.top .logo .o{width:26px;height:26px;border-radius:7px;background:linear-gradient(135deg,#21C8A6,#1B95E0);display:grid;place-items:center;font-size:.9rem}
.top .nav{display:flex;gap:4px;margin-left:18px}
.top .nav a{color:#AEC2E8;font-size:.84rem;padding:6px 12px;border-radius:7px;text-decoration:none}
.top .nav a.on{background:#16306B;color:#fff}
.top .right{margin-left:auto;display:flex;align-items:center;gap:14px;font-size:.82rem;color:#AEC2E8}
.top .pulse{width:8px;height:8px;border-radius:50%;background:#21C8A6;box-shadow:0 0 8px #21C8A6;animation:p 1.6s infinite}
@keyframes p{0%,100%{opacity:1}50%{opacity:.3}}
.wrap{padding:20px 22px}
.crumb{font-size:.8rem;color:#6B7B9E;margin-bottom:14px}
.crumb b{color:#1B2A4A}
.barbox{background:#fff;border:1px solid #DCE3EF;border-radius:12px;box-shadow:0 6px 20px rgba(30,60,120,.07);overflow:hidden}
.barhead{display:flex;align-items:center;padding:13px 18px;border-bottom:1px solid #EAEFF7}
.barhead h2{font-size:1rem;font-weight:700}
.barhead .count{margin-left:10px;background:#E8F0FF;color:#1B6FE0;font-size:.72rem;font-weight:700;padding:2px 9px;border-radius:9px}
.barhead .refresh{margin-left:auto;font-size:.74rem;color:#6B7B9E;font-family:'JetBrains Mono'}
table{width:100%;border-collapse:collapse}
thead th{text-align:left;font-size:.68rem;letter-spacing:.07em;text-transform:uppercase;color:#8493B3;padding:11px 18px;border-bottom:1px solid #EAEFF7;font-weight:700;background:#FAFBFE}
tbody td{padding:13px 18px;border-bottom:1px solid #F1F4FA;font-size:.86rem}
tbody tr{cursor:pointer;transition:background .12s}
tbody tr:hover{background:#F5F9FF}
.num{font-family:'JetBrains Mono';font-weight:600;color:#1B6FE0}
.pri{display:inline-flex;align-items:center;gap:6px;font-weight:600;font-size:.8rem}
.pri .pb{width:4px;height:16px;border-radius:2px}
```



```

.state{font-size:.72rem;font-weight:700;padding:3px 11px;border-radius:99px;display:inline-block}
.s-New{background:#FEE9EC;color:#D5304B}.s-InProgress{background:#FFF3DF;color:#B5760B}
.s-Resolved{background:#E2F6EE;color:#149563}.s-Closed{background:#EAEFF7;color:#5B6B8C}
.empty{padding:50px;text-align:center;color:#8493B3;font-style:italic}
/* drawer */
.scrim{position:fixed;inset:0;background:rgba(11,31,77,.35);display:none;z-index:40}
.scrim.on{display:block}
.drawer{position:fixed;top:0;right:-560px;width:540px;max-width:94vw;height:100%;background:#fff;
  box-shadow:-12px 0 40px rgba(11,31,77,.22);z-index:50;transition:right .28s ease;overflow-y:auto}
.drawer.on{right:0}
.dhead{background:linear-gradient(120deg,#0B1F4D,#16306B);color:#fff;padding:20px 24px}
.dhead .dn{font-family:'JetBrains Mono';font-size:1.4rem;font-weight:700}
.dhead .dt{color:#C4D4F2;font-size:.9rem;margin-top:4px}
.dhead .x{position:absolute;top:18px;right:20px;color:#C4D4F2;font-size:1.5rem;cursor:pointer;border:none;background:none}
.dbody{padding:22px 24px}
.frow{display:flex;gap:14px;margin-bottom:16px}
.fld{flex:1}
.fld label{display:block;font-size:.66rem;letter-spacing:.06em;text-transform:uppercase;color:#8493B3;font-weight:700;margin-bottom:4px}
.fld .v{font-size:.9rem;color:#1B2A4A;font-weight:500}
.sec{font-size:.7rem;letter-spacing:.08em;text-transform:uppercase;color:#1B6FE0;font-weight:700;
  margin:20px 0 10px;padding-bottom:6px;border-bottom:2px solid #EAEFF7}
.desc{background:#F7F9FD;border:1px solid #EAEFF7;border-radius:9px;padding:12px 14px;font-size:.88rem;line-height:1.5;color:#33425E}
.ci{display:inline-block;font-family:'JetBrains Mono';font-size:.74rem;background:#EEF3FB;color:#2A4A86;
  padding:3px 9px;border-radius:6px;margin:2px 4px 2px 0}
.note{padding:11px 0;border-bottom:1px solid #F1F4FA}
.note .meta{font-size:.72rem;color:#8493B3;font-family:'JetBrains Mono';margin-bottom:3px}
.note .meta b{color:#1B6FE0}
.note .txt{font-size:.86rem;color:#33425E}
.resolvebtn{margin-top:20px;width:100%;background:linear-gradient(120deg,#10B981,#059669);color:#fff;
  border:none;border-radius:11px;padding:13px;font-family:'Space Grotesk',sans-serif;font-weight:700;
  font-size:.95rem;cursor:pointer;box-shadow:0 8px 22px rgba(16,185,129,.35)}
.resolvebtn:hover{transform:translateY(-1px)}
.resolved-banner{margin-top:20px;background:#E2F6EE;color:#149563;border-radius:11px;padding:13px;
  text-align:center;font-weight:700;font-size:.9rem}
.hint{margin-top:22px;background:#EEF6FF;border-left:4px solid #1B6FE0;border-radius:0 9px 9px 0;padding:12px 15px;font-size:.82rem;color:#2A4A86}
</style></head><body>
<div class="top">
  <div class="logo"><span class="o">SD</span> ServiceDesk</div>
  <div class="nav"><a class="on" href="#">Incidents</a><a href="#">Problems</a><a href="#">Changes</a><a href="#">CMDB</a></div>
  <div class="right"><span class="pulse"></span>Live · auto-refresh 3s</div>
</div>
<div class="wrap">
  <div class="crumb">Incident Management &nbsp;&nbsp;&nbsp;<b>All Open Incidents</b></div>
  <div class="barbox">
    <div class="barhead"><h2>Incidents</h2><span class="count" id="cnt">0</span>
    <span class="refresh" id="rf">updated ?</span></div>
    <table><thead><tr>
      <th>Number</th><th>Priority</th><th>Short description</th><th>State</th><th>Assignment group</th><th>Updated</th>
    </tr></thead><tbody id="rows"><tr><td colspan="6" class="empty">No incidents yet. Trigger an outage on the NOC dashboard.</td></tr></tbody></table>
  </div>
</div>

<div class="scrim" id="scrim" onclick="closeD()"></div>
<div class="drawer" id="drawer">
  <div class="dhead" style="position:relative">
    <button class="x" onclick="closeD()">x</button>
    <div class="dn" id="d_num"></div><div class="dt" id="d_title"></div>
  </div>
  <div class="dbody" id="d_body"></div>
</div>

<script>
const PRI={ "P1": "#D5304B", "P2": "#E8820B", "P3": "#1B95E0", "P4": "#5B6B8C" };
function stateClass(s){return 's-'+(s||'New').replace(/\s+/g, '');}
async function load(){
  try{
    const r=await fetch('api/tickets'); const t=await r.json();
    document.getElementById('cnt').textContent=t.length;
    document.getElementById('rf').textContent='updated '+new Date().toLocaleTimeString();
  }
}

```

```

const tb=document.getElementById('rows');
if(!t.length){tb.innerHTML='<tr><td colspan=6 class=empty>No incidents yet. Trigger an outage on the NOC das
hboard.</td></tr>';return;}
tb.innerHTML=t.map(x=>`<tr onclick="openD('${x.ticket_id}')">
  <td class=num>${x.number}||x.ticket_id</td>
  <td><span class=pri><span class=pb style="background:${PRI[x.priority]}||'#888'"></span>${x.priority}||'|'<
/span></td>
  <td>${x.short_description}||x.title||'|'</td>
  <td><span class="state ${stateClass(x.status)}">${x.status}||'New'</span></td>
  <td>${x.assignment_group}||'|'</td>
  <td style="color:#6B7B9E;font-family:JetBrains Mono;font-size:.78rem">${(x.updated_at||'|').slice(11)}||x.u
pdated_at||'|'</td>
</tr>`).join('');
}catch(e){}
}
async function openD(id){
  const r=await fetch('api/ticket/'+id); const x=await r.json();
  document.getElementById('d_num').textContent=x.number||x.ticket_id;
  document.getElementById('d_title').textContent=x.short_description||x.title||'|';
  const cis=(x.affected_devices||[]).map(d=>`<span class=ci>${d}</span>`).join('')||'|<span style="color:#8493B3"
>none</span>';
  const notes=(x.work_notes||[]).slice().reverse().map(n=>`<div class=note><div class=meta><b>${n.author}</b> .
${n.ts}</div><div class=txt>${n.text}</div></div>`).join('')||'|<span style="color:#8493B3">No work notes.</span>
';
  document.getElementById('d_body').innerHTML=`
    <div class=frow>
      <div class=fld><label>State</label><div class=v><span class="state ${stateClass(x.status)}">${x.status}||'N
ew'</span></div></div>
      <div class=fld><label>Priority</label><div class=v>${x.priority}||'|'</div></div>
      <div class=fld><label>Category</label><div class=v>${x.category}||'Network'</div></div>
    </div>
    <div class=frow>
      <div class=fld><label>Assigned to</label><div class=v>${x.assigned_to}||'? unassigned ?'</div></div>
      <div class=fld><label>Assignment group</label><div class=v>${x.assignment_group}||'|'</div></div>
    </div>
    <div class=sec>Description</div><div class=desc>${x.description}||'|'</div>
    <div class=sec>Root cause</div><div class=desc>${x.root_cause}||'Pending analysis'</div>
    <div class=sec>Affected CIs</div><div>${cis}</div>
    <div class=sec>Work notes &amp; activity</div>${notes}
    ${x.status==='Resolved'||x.status==='Closed'
      ? '<div class=resolved-banner>&#10004; This incident is '+x.status+'</div>'
      : '<button class=resolvebtn id=rbtn>&#10004; Resolve this incident</button>'}';
  var _rb=document.getElementById('rbtn');
  if(_rb){_rb.onclick=function(){resolveTicket(x.ticket_id);}}
  document.getElementById('scrim').classList.add('on');
  document.getElementById('drawer').classList.add('on');
}
async function resolveTicket(id){
  await fetch('api/resolve/'+id,{method:'POST'});
  openD(id);
  load();
}
function closed(){document.getElementById('scrim').classList.remove('on');document.getElementById('drawer').clas
sList.remove('on');}
load(); setInterval(load,3000);
</script></body></html>""

```

```

@app.post("/api/resolve/{tid}")
def api_resolve(tid: str):
    import requests as _rq
    try:
        note = ("Incident resolved from ITSM board. Remediation actions applied; "
                "connectivity restored and alarms cleared.")
        r = _rq.post(f"{ITSM_API_URL}/update_ticket",
                    json={"ticket_id": tid, "status": "Resolved", "work_note": note}, timeout=5)
        return JSONResponse(r.json())
    except Exception as e:
        return JSONResponse({"error": str(e)}, status_code=200)

```

```

@app.get("/", response_class=HTMLResponse)
def board():
    return PAGE

```

```
if __name__ == "__main__":  
    uvicorn.run(app, host="0.0.0.0", port=ITSM_BOARD_PORT)
```

admin_panel.py

```
# admin_panel.py - Standalone Admin / Configuration panel (port 8090).
# Edit SLA + notification recipients. Reads/writes settings.py (JSON-backed).
import uvicorn
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse, JSONResponse
import settings as st

app = FastAPI(title="NOC Admin")
ADMIN_PORT = 8090

@app.get("/api/settings")
def api_settings():
    return JSONResponse(st.load_settings())

@app.post("/api/sla")
async def api_sla(req: Request):
    body = await req.json()
    try:
        secs = int(body.get("sla_seconds", 300))
        st.save_settings({"sla_seconds": max(30, secs)})
    except Exception as e:
        return JSONResponse({"error": str(e)}, status_code=200)
    return JSONResponse(st.load_settings())

@app.post("/api/recipient/add")
async def api_add(req: Request):
    b = await req.json()
    st.add_recipient(b.get("team", ""), b.get("channel", "email"), b.get("contact", ""))
    return JSONResponse(st.load_settings())

@app.post("/api/recipient/remove")
async def api_remove(req: Request):
    b = await req.json()
    try:
        st.remove_recipient(int(b.get("index", -1)))
    except Exception:
        pass
    return JSONResponse(st.load_settings())

PAGE = """<!doctype html><html><head><meta charset=utf-8>
<meta name=viewport content="width=device-width, initial-scale=1">
<title>NOC Admin · Configuration</title>
<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=JetBrains+Mono:wght@400;
600&family=Space+Gothic:wght@600;700&display=swap');
*{margin:0;padding:0;box-sizing:border-box}
body{font-family:Inter,sans-serif;background:#EEF2FB;color:#1E293B;padding:0}
.top{background:linear-gradient(120deg,#6366F1,#8B5CF6,#06B6D4);color:#fff;padding:20px 28px}
.top h1{font-family:'Space Gothic';font-size:1.4rem}
.top p{color:#E0E7FF;font-size:.85rem;font-family:'JetBrains Mono';margin-top:4px}
.wrap{max-width:880px;margin:22px auto;padding:0 20px;display:grid;gap:18px}
.card{background:#fff;border:1px solid #E2E8F5;border-radius:16px;padding:22px 24px;box-shadow:0 10px 30px rgba(
80,100,200,.10)}
.card h2{font-family:'Space Gothic';font-size:1.05rem;margin-bottom:14px;display:flex;align-items:center;gap:8p
x}
label{font-size:.72rem;text-transform:uppercase;letter-spacing:.06em;color:#64748B;font-weight:700;display:block
;margin-bottom:6px}
input,select{font-family:Inter;font-size:.9rem;padding:9px 12px;border:1px solid #D8E0F0;border-radius:9px;width
:100%}
.row{display:flex;gap:12px;align-items:flex-end;flex-wrap:wrap}
.row>div{flex:1;min-width:140px}
button{font-family:'Space Gothic';font-weight:700;color:#fff;border:none;border-radius:10px;padding:10px 18px;c
ursor:pointer;
    background:linear-gradient(120deg,#6366F1,#8B5CF6);box-shadow:0 6px 18px rgba(99,102,241,.35)}
button.del{background:#FEE2E2;color:#DC2626;box-shadow:none;padding:7px 12px;font-size:.8rem}
.slabox{display:flex;align-items:center;gap:14px}
.slabox .cur{font-family:'JetBrains Mono';font-size:1.6rem;font-weight:700;color:#6366F1}
table{width:100%;border-collapse:collapse;margin-top:6px}
th{text-align:left;font-size:.66rem;text-transform:uppercase;letter-spacing:.06em;color:#94A3B8;padding:8px 10px
;border-bottom:1px solid #EEF2FB}
td{padding:10px;border-bottom:1px solid #F4F7FD;font-size:.88rem}
.chan{font-family:'JetBrains Mono';font-size:.7rem;font-weight:700;padding:2px 9px;border-radius:6px}
.c-page{background:#FEE2E2;color:#DC2626}.c-email{background:#EEF2FF;color:#4F46E5}.c-slack{background:#D1FAE5;c
```

```

olor:#059669}
.ok{color:#059669;font-size:.8rem;font-weight:600;margin-left:10px}
</style></head><body>
<div class=top><h1>&#9881;&#65039; NOC Copilot &middot; Admin Configuration</h1>
<p>Live operational settings &middot; changes apply to new incident simulations</p></div>
<div class=wrap>
  <div class=card>
    <h2>&#9201;&#65039; SLA Timer</h2>
    <div class=slabox>
      <div style="flex:0 0 auto"><label>Current SLA window</label><span class=cur id=slacur>--</span></div>
      <div style="flex:1"><label>Set SLA (seconds)</label><input id=slainp type=number min=30 value=300></div>
      <button onclick=saveSLA()>Save SLA</button><span class=ok id=slaok></span>
    </div>
  </div>
  <div class=card>
    <h2>&#128226; Notification Manager</h2>
    <table><thead><tr><th>Team</th><th>Channel</th><th>Contact</th><th></th></tr></thead>
    <tbody id=recips></tbody></table>
    <div class=row style="margin-top:16px">
      <div><label>Team / Role</label><input id=nteam placeholder="e.g. Field Ops"></div>
      <div><label>Channel</label><select id=nchan><option>email</option><option>slack</option><option>page</option></select></div>
      <div><label>Contact</label><input id=ncontact placeholder="email / #channel"></div>
      <button onclick=addRecip()>Add</button>
    </div>
  </div>
</div>
<script>
async function load(){
  const r=await fetch('api/settings'); const s=await r.json();
  document.getElementById('slacur').textContent=(s.sla_seconds/60).toFixed(1)+' min';
  document.getElementById('slainp').value=s.sla_seconds;
  const tb=document.getElementById('recips');
  tb.innerHTML=(s.notification_recipients||[]).map((x,i)=>`<tr>
    <td><b>${x.team}</b></td>
    <td><span class="chan c-${x.channel}">${x.channel.toUpperCase()}</span></td>
    <td style="font-family:JetBrains Mono;font-size:.82rem">${x.contact}</td>
    <td><button class=del onclick=delRecip(${i})>Remove</button></td></tr>`).join('');
}
async function saveSLA(){
  const v=parseInt(document.getElementById('slainp').value);
  await fetch('api/sla',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({sla_seconds:v})});
  document.getElementById('slaok').textContent='? saved'; setTimeout(()=>document.getElementById('slaok').textContent='',1500); load();
}
async function addRecip(){
  const team=document.getElementById('nteam').value, channel=document.getElementById('nchan').value, contact=document.getElementById('ncontact').value;
  if(!team)return;
  await fetch('api/recipient/add',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({team,channel,contact})});
  document.getElementById('nteam').value='';document.getElementById('ncontact').value=''; load();
}
async function delRecip(i){
  await fetch('api/recipient/remove',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify({index:i})}); load();
}
load();
</script></body></html>""

@app.get("/", response_class=HTMLResponse)
def home():
    return PAGE

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=ADMIN_PORT)

```

run_all.sh

```
#!/bin/bash
cd /workspace/shared/noc-copilot
echo "==> Stopping old app processes (vLLM on :8000 untouched)..."
pkill -9 -f "router_api" 2>/dev/null
pkill -9 -f "itsm_api" 2>/dev/null
pkill -9 -f "itsm_board" 2>/dev/null
pkill -9 -f "admin_panel" 2>/dev/null
pkill -9 -f "app.py" 2>/dev/null
fuser -k -9 8001/tcp 8002/tcp 8080/tcp 8090/tcp 7860/tcp 2>/dev/null
sleep 4
echo "==> Starting background services..."
uvicorn mocks.router_api:app --host 0.0.0.0 --port 8001 > /tmp/router.log 2>&1 &
uvicorn mocks.itsm_api:app --host 0.0.0.0 --port 8002 > /tmp/itsm.log 2>&1 &
python itsm_board.py > /tmp/board.log 2>&1 &
python admin_panel.py > /tmp/admin.log 2>&1 &
sleep 4
echo "==> Health checks:"
curl -s http://localhost:8001/docs >/dev/null && echo " router :8001 OK" || echo " router :8001 FAIL"
curl -s http://localhost:8002/health && echo " <- itsm :8002 OK" || echo " itsm :8002 FAIL"
curl -s http://localhost:8080/ >/dev/null && echo " board :8080 OK" || echo " board :8080 FAIL"
curl -s http://localhost:8090/ >/dev/null && echo " admin :8090 OK" || echo " admin :8090 FAIL"
echo ""
echo "==> ITSM board: https://notebooks.amd.com/jupyter-hack-team-3049-260614162200-ee4e0c4e/proxy/8080/"
echo "==> Admin panel: https://notebooks.amd.com/jupyter-hack-team-3049-260614162200-ee4e0c4e/proxy/8090/"
echo ""
echo "==> Starting dashboard..."
python app.py
```

data/alarm_generator.py

```
# data/alarm_generator.py
# Generates synthetic telecom NOC alarms. The hero scenario is generate_storm():
# ONE root fault on the core router cascades into ~40 downstream alarms.
# The model reasons ABOUT this data; the generator just produces realistic events.

import json
import os
import random
from datetime import datetime, timedelta

_HERE = os.path.dirname(os.path.abspath(__file__))
TOPOLOGY_FILE = os.path.join(_HERE, "topology.json")

def load_topology():
    with open(TOPOLOGY_FILE) as f:
        return json.load(f)

def _descendants(topo, device_id):
    """All devices topologically below device_id (children, grandchildren...)."""
    out = []
    stack = list(topo["devices"][device_id]["downstream"])
    while stack:
        d = stack.pop()
        out.append(d)
        stack.extend(topo["devices"][d]["downstream"])
    return out

def _alarm(aid, ts, device_id, topo, alarm_type, severity, description, kpis):
    dev = topo["devices"][device_id]
    return {
        "alarm_id": aid,
        "timestamp": ts.isoformat(),
        "device_id": device_id,
        "device_type": dev["type"],
        "site": dev["site"],
        "alarm_type": alarm_type,
        "severity": severity,
        "description": description,
        "kpis": kpis,
    }

def generate_storm(seed=42):
    """The scripted demo scenario.

    Root fault: Router-Core-01 BGP peering collapses (CRITICAL) at t=0.
    Cascade: every downstream PE router, aggregation switch and base station
    raises symptomatic alarms (packet loss, link down, high latency,
    sync loss) over the following ~60 seconds.

    Returns a list of ~40 alarms, root first, ordered by time.
    """
    random.seed(seed)
    topo = load_topology()
    t0 = datetime(2026, 6, 16, 9, 0, 0)
    alarms = []
    n = 1

    # 1) ROOT EVENT
    alarms.append(_alarm(
        f"ALM-{n:03d}", t0, "Router-Core-01", topo,
        "BGP_PEER_DOWN", "CRITICAL",
        "BGP session lost with upstream peer 10.0.0.1; route table withdrawal in progress",
        {"cpu": 94, "packet_loss": 0, "latency_ms": 0},
    ))
    n += 1

    affected = _descendants(topo, "Router-Core-01")

    # 2) PE routers react first (~10-25s)
```

```

for dev in [d for d in affected if topo["devices"][d]["type"] == "pe"]:
    offset = random.randint(8, 25)
    for atype, sev, desc, kpis in [
        ("BGP_PEER_DOWN", "CRITICAL",
         f"BGP session to Router-Core-01 down; lost default route",
         {"cpu": random.randint(70, 88), "packet_loss": random.randint(40, 80), "latency_ms": random.randint
(120, 400)}),
        ("PACKET_LOSS", "MAJOR",
         f"Elevated packet loss on uplink toward core",
         {"cpu": random.randint(40, 60), "packet_loss": random.randint(30, 70), "latency_ms": random.randint
(80, 300)}),
    ]:
        alarms.append(_alarm(f"ALM-{n:03d}", t0 + timedelta(seconds=offset), dev, topo, atype, sev, desc, kp
is))
        n += 1
        offset += random.randint(1, 4)

# 3) Aggregation switches (~20-40s)
for dev in [d for d in affected if topo["devices"][d]["type"] == "switch"]:
    offset = random.randint(20, 40)
    for atype, sev, desc, kpis in [
        ("LINK_DOWN", "MAJOR",
         f"Uplink interface to PE router went down",
         {"cpu": random.randint(20, 45), "packet_loss": random.randint(50, 100), "latency_ms": 0}),
        ("HIGH_LATENCY", "MINOR",
         f"Round-trip latency above threshold on aggregation uplink",
         {"cpu": random.randint(20, 40), "packet_loss": random.randint(10, 40), "latency_ms": random.randint
(200, 600)}),
    ]:
        alarms.append(_alarm(f"ALM-{n:03d}", t0 + timedelta(seconds=offset), dev, topo, atype, sev, desc, kp
is))
        n += 1
        offset += random.randint(1, 5)

# 4) Base stations lose service (~35-60s) - two symptoms each
for dev in [d for d in affected if topo["devices"][d]["type"] == "bts"]:
    offset = random.randint(35, 55)
    for atype, sev, desc, kpis in [
        ("SERVICE_DEGRADED", "MAJOR",
         "Cell site lost backhaul connectivity; subscribers dropping",
         {"cpu": random.randint(10, 30), "packet_loss": random.randint(60, 100), "latency_ms": random.ran
t(300, 900)}),
        ("SYNC_LOSS", "MINOR",
         "Loss of timing synchronisation from backhaul",
         {"cpu": random.randint(10, 25), "packet_loss": random.randint(40, 90), "latency_ms": random.randint
(200, 700)}),
    ]:
        alarms.append(_alarm(f"ALM-{n:03d}", t0 + timedelta(seconds=offset), dev, topo, atype, sev, desc, kp
is))
        n += 1
        offset += random.randint(1, 4)

alarms.sort(key=lambda a: a["timestamp"])
return alarms

```

```

def generate_scenario(scenario="p1", seed=None):
    import random as _r
    if seed is not None:
        _r.seed(seed)
    if scenario == "p1":
        return generate_storm(seed=seed if seed is not None else 42)
    topo = load_topology()
    t0 = datetime(2026, 6, 16, 9, 0, 0)
    pe = [d for d in topo["devices"] if topo["devices"][d]["type"] == "pe"]
    sw = [d for d in topo["devices"] if topo["devices"][d]["type"] == "switch"]
    bts = [d for d in topo["devices"] if topo["devices"][d]["type"] == "bts"]
    alts = {
        "fiber_cut": {"root_dev": (_r.choice(pe) if pe else "PE-Router-07"),
                      "root_type": "LINK_DOWN", "root_sev": "MAJOR",
                      "root_desc": "Fiber uplink cut detected; interface down, no carrier signal",
                      "root_kpis": {"cpu": 30, "packet_loss": 100, "latency_ms": 0},
                      "child_type": "PACKET_LOSS", "child_sev": "MINOR",
                      "child_desc": "Increased packet loss on rerouted path"},
        "cpu_exhaustion": {"root_dev": (_r.choice(sw) if sw else "Switch-AGG-03"),
                           "root_type": "HIGH_CPU", "root_sev": "MAJOR",

```



```

        "root_desc": "Control-plane CPU sustained at 97%; forwarding degraded",
        "root_kpis": {"cpu": 97, "packet_loss": 15, "latency_ms": 220},
        "child_type": "HIGH_LATENCY", "child_sev": "MINOR",
        "child_desc": "Latency rising on switched paths due to CPU saturation"},
    "bts_sync": {"root_dev": (_r.choice(bts) if bts else "BTS-1021"),
        "root_type": "SYNC_LOSS", "root_sev": "MINOR",
        "root_desc": "Loss of timing synchronisation at cell site; GPS reference unstable",
        "root_kpis": {"cpu": 18, "packet_loss": 5, "latency_ms": 60},
        "child_type": "SERVICE_DEGRADED", "child_sev": "MINOR",
        "child_desc": "Minor service degradation at single cell site"},
    "port_flap": {"root_dev": (_r.choice(sw) if sw else "Switch-AGG-04"),
        "root_type": "INTERFACE_FLAP", "root_sev": "MINOR",
        "root_desc": "Access port flapping repeatedly; intermittent link",
        "root_kpis": {"cpu": 25, "packet_loss": 8, "latency_ms": 40},
        "child_type": "HIGH_LATENCY", "child_sev": "MINOR",
        "child_desc": "Brief latency spikes during flap events"},
    }
if scenario == "random":
    scenario = _r.choice(list(alts.keys()))
    cfg = alts.get(scenario, alts["fiber_cut"])
    alarms = []; n = 1
    alarms.append(_alarm(f"ALM-{n:03d}", t0, cfg["root_dev"], topo,
        cfg["root_type"], cfg["root_sev"], cfg["root_desc"], cfg["root_kpis"]))

    n += 1
    neighbours = (topo["devices"][cfg["root_dev"]].get("downstream") or [])[:4]
    for dev in neighbours:
        off = _r.randint(5, 25)
        alarms.append(_alarm(f"ALM-{n:03d}", t0 + timedelta(seconds=off), dev, topo,
            cfg["child_type"], cfg["child_sev"], cfg["child_desc"],
            {"cpu": _r.randint(20, 60), "packet_loss": _r.randint(5, 40),
             "latency_ms": _r.randint(40, 300)}))

        n += 1
    alarms.sort(key=lambda a: a["timestamp"])
    return alarms

def generate_normal_alarms(n=5, seed=None):
    """A few unrelated, low-priority alarms (no common root) for contrast."""
    if seed is not None:
        random.seed(seed)
    topo = load_topology()
    devices = list(topo["devices"].keys())
    types = [
        ("HIGH_CPU", "MINOR", "CPU utilisation above 80% threshold"),
        ("INTERFACE_FLAP", "WARNING", "Interface flapped once and recovered"),
        ("CONFIG_CHANGE", "WARNING", "Configuration change committed by operator"),
        ("TEMPERATURE_WARN", "MINOR", "Chassis temperature above nominal"),
    ]
    t0 = datetime(2026, 6, 16, 8, 0, 0)
    out = []
    for i in range(n):
        dev = random.choice(devices)
        atype, sev, desc = random.choice(types)
        out.append(_alarm(
            f"NRM-{i+1:03d}", t0 + timedelta(minutes=random.randint(0, 50)),
            dev, topo, atype, sev, desc,
            {"cpu": random.randint(20, 85), "packet_loss": random.randint(0, 5), "latency_ms": random.randint(1,
30)}),
        ))
    return out

if __name__ == "__main__":
    storm = generate_storm()
    print(f"Generated {len(storm)} alarms in the storm scenario.\n")
    print("Root event:")
    print(json.dumps(storm[0], indent=2))
    print("\nFirst 3 cascading alarms:")
    for a in storm[1:4]:
        print(f"  {a['alarm_id']} | {a['timestamp'][11:19]} | {a['device_id']:16} | {a['severity']:8} | {a['alarm_type']}")
    print(f"\nDevices involved: {len(set(a['device_id'] for a in storm))}")
    assert 30 <= len(storm) <= 45, f"expected ~40 alarms, got {len(storm)}"
    assert storm[0]["device_id"] == "Router-Core-01", "root should be the core router"
    print("Alarm generator test passed.")

```

data/topology.json

```
{
  "devices": {
    "Router-Core-01": {
      "type": "core",
      "site": "DC-North",
      "role": "Core BGP Router",
      "upstream": null,
      "downstream": ["PE-Router-07", "PE-Router-09", "PE-Router-11"]
    },
    "PE-Router-07": {
      "type": "pe",
      "site": "Metro-East",
      "role": "Provider Edge Router",
      "upstream": "Router-Core-01",
      "downstream": ["Switch-AGG-03", "Switch-AGG-04"]
    },
    "PE-Router-09": {
      "type": "pe",
      "site": "Metro-West",
      "role": "Provider Edge Router",
      "upstream": "Router-Core-01",
      "downstream": ["Switch-AGG-05", "Switch-AGG-06"]
    },
    "PE-Router-11": {
      "type": "pe",
      "site": "Metro-South",
      "role": "Provider Edge Router",
      "upstream": "Router-Core-01",
      "downstream": ["Switch-AGG-07"]
    },
    "Switch-AGG-03": {
      "type": "switch",
      "site": "Metro-East",
      "role": "Aggregation Switch",
      "upstream": "PE-Router-07",
      "downstream": ["BTS-1021", "BTS-1022"]
    },
    "Switch-AGG-04": {
      "type": "switch",
      "site": "Metro-East",
      "role": "Aggregation Switch",
      "upstream": "PE-Router-07",
      "downstream": ["BTS-1023"]
    },
    "Switch-AGG-05": {
      "type": "switch",
      "site": "Metro-West",
      "role": "Aggregation Switch",
      "upstream": "PE-Router-09",
      "downstream": ["BTS-1031", "BTS-1032"]
    },
    "Switch-AGG-06": {
      "type": "switch",
      "site": "Metro-West",
      "role": "Aggregation Switch",
      "upstream": "PE-Router-09",
      "downstream": ["BTS-1033"]
    },
    "Switch-AGG-07": {
      "type": "switch",
      "site": "Metro-South",
      "role": "Aggregation Switch",
      "upstream": "PE-Router-11",
      "downstream": ["BTS-1041", "BTS-1042"]
    },
    "BTS-1021": {"type": "bts", "site": "Metro-East", "role": "Base Station", "upstream": "Switch-AGG-03", "downstream": []},
    "BTS-1022": {"type": "bts", "site": "Metro-East", "role": "Base Station", "upstream": "Switch-AGG-03", "downstream": []},
    "BTS-1023": {"type": "bts", "site": "Metro-East", "role": "Base Station", "upstream": "Switch-AGG-04", "downstream": []},
    "BTS-1031": {"type": "bts", "site": "Metro-West", "role": "Base Station", "upstream": "Switch-AGG-05", "downstream": []},
    "BTS-1032": {"type": "bts", "site": "Metro-West", "role": "Base Station", "upstream": "Switch-AGG-05", "downstream": []},
    "BTS-1033": {"type": "bts", "site": "Metro-West", "role": "Base Station", "upstream": "Switch-AGG-06", "downstream": []},
    "BTS-1041": {"type": "bts", "site": "Metro-South", "role": "Base Station", "upstream": "Switch-AGG-07", "downstream": []},
    "BTS-1042": {"type": "bts", "site": "Metro-South", "role": "Base Station", "upstream": "Switch-AGG-07", "downstream": []}
  }
}
```

```
    "BTS-1032": {"type": "bts", "site": "Metro-West", "role": "Base Station", "upstream": "Switch-AGG-05", "downstream": []},
    "BTS-1033": {"type": "bts", "site": "Metro-West", "role": "Base Station", "upstream": "Switch-AGG-06", "downstream": []},
    "BTS-1041": {"type": "bts", "site": "Metro-South", "role": "Base Station", "upstream": "Switch-AGG-07", "downstream": []},
    "BTS-1042": {"type": "bts", "site": "Metro-South", "role": "Base Station", "upstream": "Switch-AGG-07", "downstream": []}
  }
}
```

mocks/itsm_api.py

```
# mocks/itsm_api.py - ServiceNow-style mock ITSM API (upgraded)
# Keeps the original create_ticket / update_ticket contract used by pipeline.py,
# and ADDS board endpoints (list, get, work-notes history, flexible update).

from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse
from pydantic import BaseModel
from typing import List, Optional, Dict, Any
import uuid
from datetime import datetime

app = FastAPI(title="Mock ITSM (ServiceNow-style)")

# In-memory ticket store
tickets_db: Dict[str, Dict[str, Any]] = {}

def _now():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# ----- request models -----
class CreateTicketRequest(BaseModel):
    priority: str
    title: str
    description: str
    affected_devices: Optional[List[str]] = None
    root_cause: Optional[str] = None

class UpdateTicketRequest(BaseModel):
    ticket_id: str
    status: Optional[str] = None
    work_note: Optional[str] = None
    # full control: any of these may be set from chat
    priority: Optional[str] = None
    assigned_to: Optional[str] = None
    assignment_group: Optional[str] = None
    category: Optional[str] = None

# ----- create -----
@app.post("/create_ticket")
async def create_ticket(req: CreateTicketRequest):
    ticket_id = f"INC{str(uuid.uuid4().int)[:7]}"
    now = _now()
    tickets_db[ticket_id] = {
        "ticket_id": ticket_id,
        "number": ticket_id,
        "status": "New",
        "priority": req.priority,
        "title": req.title,
        "short_description": req.title,
        "description": req.description,
        "affected_devices": req.affected_devices or [],
        "root_cause": req.root_cause,
        "assigned_to": "? unassigned ?",
        "assignment_group": "Network Operations",
        "category": "Network",
        "created_at": now,
        "updated_at": now,
        "url": f"https://itsm.mock/{ticket_id}",
        "work_notes": [
            {"ts": now, "author": "NOC Copilot", "text": "Incident auto-created by NOC Agentic Copilot."}
        ],
    }
    return tickets_db[ticket_id]

# ----- flexible update (full control) -----
@app.post("/update_ticket")
async def update_ticket(req: UpdateTicketRequest):
    if req.ticket_id not in tickets_db:
```

```

        raise HTTPException(status_code=404, detail="Ticket not found")
    t = tickets_db[req.ticket_id]
    changed = []
    for field in ["status", "priority", "assigned_to", "assignment_group", "category"]:
        val = getattr(req, field)
        if val:
            t[field] = val
            changed.append(f"{field} ? {val}")
    if req.work_note:
        t["work_notes"].append({"ts": _now(), "author": "NOC Copilot", "text": req.work_note})
        changed.append("work note added")
    t["updated_at"] = _now()
    if changed:
        t["work_notes"].append({"ts": _now(), "author": "system", "text": "Updated: " + "; ".join(changed)})
    return t

# ----- board endpoints -----
@app.get("/tickets")
async def list_tickets():
    # newest first
    return sorted(tickets_db.values(), key=lambda x: x["created_at"], reverse=True)

@app.get("/ticket/{ticket_id}")
async def get_ticket(ticket_id: str):
    if ticket_id not in tickets_db:
        raise HTTPException(status_code=404, detail="Ticket not found")
    return tickets_db[ticket_id]

@app.get("/health")
async def health():
    return {"status": "ok", "tickets": len(tickets_db)}

```

mocks/router_api.py

```
from fastapi import FastAPI, HTTPException
import time
import random
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class ClearBGPRequest(BaseModel):
    device_id: str
    peer: str

class RestartInterfaceRequest(BaseModel):
    device_id: str
    interface: str

class ResetRouterRequest(BaseModel):
    device_id: str

class ClearBGPResponse(BaseModel):
    status: str
    device_id: str
    action: str
    device_response: str
    latency_ms: int

class RestartInterfaceResponse(BaseModel):
    status: str
    device_id: str
    action: str
    device_response: str
    latency_ms: int

class ResetRouterResponse(BaseModel):
    status: str
    device_id: str
    action: str
    device_response: str
    latency_ms: int

@app.post("/clear_bgp", response_model=ClearBGPResponse)
async def clear_bgp(request: ClearBGPRequest):
    # Simulate ~0.5s latency
    time.sleep(0.5)

    # Return realistic response
    return ClearBGPResponse(
        status="SUCCESS",
        device_id=request.device_id,
        action="clear_bgp",
        device_response="BGP session reset, neighbor re-established",
        latency_ms=480
    )

@app.post("/restart_interface", response_model=RestartInterfaceResponse)
async def restart_interface(request: RestartInterfaceRequest):
    # Simulate ~0.5s latency
    time.sleep(0.5)

    # Occasionally fail (simulate 20% failure rate)
    if random.random() < 0.2:
        raise HTTPException(status_code=500, detail="Interface restart failed due to device unresponsiveness")

    # Return realistic response
    return RestartInterfaceResponse(
        status="SUCCESS",
        device_id=request.device_id,
        action="restart_interface",
        device_response=f"Interface {request.interface} bounced successfully",
        latency_ms=510
    )

@app.post("/reset_router", response_model=ResetRouterResponse)
```

```
async def reset_router(request: ResetRouterRequest):
    # Simulate ~0.5s latency
    time.sleep(0.5)

    # Return realistic response
    return ResetRouterResponse(
        status="SUCCESS",
        device_id=request.device_id,
        action="reset_router",
        device_response="Router reboot initiated",
        latency_ms=490
    )
```

rag/knowledge_base.py

```
# rag/knowledge_base.py - RAG implementation with ChromaDB and sentence-transformers
import os
import chromadb
from chromadb.config import Settings
from sentence_transformers import SentenceTransformer
from typing import List, Dict, Any
import json

# Initialize the embedding model
EMBEDDING_MODEL = "all-MiniLM-L6-v2"

class KnowledgeBase:
    def __init__(self, persist_directory: str = "./chroma_db"):
        """
        Initialize the knowledge base with ChromaDB and sentence transformers.

        Args:
            persist_directory: Directory to store ChromaDB data
        """
        self.persist_directory = persist_directory

        # Create ChromaDB client
        self.client = chromadb.PersistentClient(
            path=persist_directory,
            settings=Settings(anonymized_telemetry=False)
        )

        # Create collection - we'll use the default embedding function
        try:
            self.collection = self.client.get_or_create_collection(name="runbooks")
        except Exception:
            # If that fails, create with explicit settings
            self.collection = self.client.create_collection(name="runbooks")

        # Create sample runbooks
        self.sample_runbooks = self._create_sample_runbooks()
        self._initialize_database()

    def _create_sample_runbooks(self) -> List[Dict[str, Any]]:
        """Create 6-8 sample runbooks for the telecom NOC context."""
        return [
            {
                "id": "bgp_peer_down",
                "title": "BGP Peer Down",
                "symptom": "BGP peer goes down",
                "possible_causes": [
                    "Physical link failure",
                    "Configuration mismatch",
                    "Resource exhaustion on router"
                ],
                "resolution_steps": [
                    "Check physical connectivity",
                    "Verify BGP configuration",
                    "Monitor resource usage",
                    "Clear BGP session if appropriate"
                ],
                "category": "network",
                "severity": "critical"
            },
            {
                "id": "router_reboot",
                "title": "Router Reboot Required",
                "symptom": "Router unresponsive or crashing",
                "possible_causes": [
                    "Software bug",
                    "Resource exhaustion",
                    "Hardware failure"
                ],
                "resolution_steps": [
                    "Check system logs",
                    "Verify uptime and resource usage",
                    "Perform graceful reboot",
                    "Document the event"
                ]
            }
        ]
```



```

    ],
    "category": "hardware",
    "severity": "critical"
  },
  {
    "id": "packet_loss",
    "title": "Packet Loss Detected",
    "symptom": "High packet loss on network links",
    "possible_causes": [
      "Link congestion",
      "Interface hardware issues",
      "Routing problems"
    ],
    "resolution_steps": [
      "Check interface statistics",
      "Monitor traffic patterns",
      "Restart interface if needed",
      "Investigate underlying routing issues"
    ],
    "category": "network",
    "severity": "major"
  },
  {
    "id": "link_down",
    "title": "Link Down",
    "symptom": "Network link goes down",
    "possible_causes": [
      "Physical cable disconnection",
      "Interface hardware failure",
      "Power outage"
    ],
    "resolution_steps": [
      "Check physical connections",
      "Verify interface status",
      "Restart interface if needed",
      "Check for hardware failures"
    ],
    "category": "hardware",
    "severity": "major"
  },
  {
    "id": "cpu_utilization_high",
    "title": "High CPU Utilization",
    "symptom": "Router CPU utilization exceeds threshold",
    "possible_causes": [
      "CPU-intensive process",
      "Memory leak",
      "Network traffic spike"
    ],
    "resolution_steps": [
      "Identify high CPU processes",
      "Monitor memory usage",
      "Clear cache if applicable",
      "Consider load balancing"
    ],
    "category": "capacity",
    "severity": "major"
  },
  {
    "id": "memory_exhaustion",
    "title": "Memory Exhaustion",
    "symptom": "Router memory usage near capacity",
    "possible_causes": [
      "Memory leak in software",
      "Insufficient memory allocation",
      "Large traffic bursts"
    ],
    "resolution_steps": [
      "Check memory usage patterns",
      "Restart memory-hungry processes",
      "Clear buffers and caches",
      "Consider hardware upgrade"
    ],
    "category": "capacity",
    "severity": "major"
  },

```

```

{
    "id": "bgp_session_flap",
    "title": "BGP Session Flapping",
    "symptom": "BGP session repeatedly goes up and down",
    "possible_causes": [
        "Unstable link",
        "Configuration inconsistency",
        "Timing issues in session negotiation"
    ],
    "resolution_steps": [
        "Check link stability",
        "Verify BGP configurations",
        "Adjust session timers if needed",
        "Clear sessions and monitor"
    ],
    "category": "network",
    "severity": "minor"
},
{
    "id": "dhcp_server_outage",
    "title": "DHCP Server Outage",
    "symptom": "DHCP service unavailable for clients",
    "possible_causes": [
        "DHCP service crash",
        "Network connectivity issues",
        "Configuration errors"
    ],
    "resolution_steps": [
        "Restart DHCP service",
        "Check network connectivity",
        "Verify configuration files",
        "Monitor lease assignments"
    ],
    "category": "software",
    "severity": "minor"
}
]

```

```

def _initialize_database(self):
    """Initialize the database with sample runbooks."""
    try:
        # Check if collection is empty
        if self.collection.count() == 0:
            print("Initializing knowledge base with sample runbooks...")

        # Convert runbooks to text format for indexing
        documents = []
        metadatas = []
        ids = []

        for runbook in self.sample_runbooks:
            # Create a rich text document combining all information
            doc_text = f"""
            Title: {runbook['title']}
            Symptom: {runbook['symptom']}
            Category: {runbook['category']}
            Severity: {runbook['severity']}

            Possible Causes:
            {' '.join(f'- {cause}' for cause in runbook['possible_causes'])}

            Resolution Steps:
            {' '.join(f'- {step}' for step in runbook['resolution_steps'])}
            """

            documents.append(doc_text.strip())
            metadatas.append({
                "title": runbook['title'],
                "symptom": runbook['symptom'],
                "category": runbook['category'],
                "severity": runbook['severity'],
                "id": runbook['id']
            })
            ids.append(runbook['id'])

        # Add documents to ChromaDB (if collection is empty)
    
```

```

        if len(documents) > 0:
            self.collection.add(
                documents=documents,
                metadatas=metadatas,
                ids=ids
            )
            print(f"Added {len(self.sample_runbooks)} sample runbooks to knowledge base")
        else:
            print(f"Knowledge base already initialized with {self.collection.count()} runbooks")

    except Exception as e:
        print(f"Error initializing knowledge base: {e}")

def add_runbook(self, title: str, symptom: str, possible_causes: List[str],
                resolution_steps: List[str], category: str, severity: str) -> str:
    """
    Add a new runbook to the knowledge base.

    Args:
        title: Runbook title
        symptom: Problem symptom
        possible_causes: List of possible causes
        resolution_steps: List of resolution steps
        category: Runbook category (network, hardware, etc.)
        severity: Severity level

    Returns:
        ID of the added runbook
    """
    # Create document text
    doc_text = f"""
    Title: {title}
    Symptom: {symptom}
    Category: {category}
    Severity: {severity}

    Possible Causes:
    {' '.join(f'- {cause}' for cause in possible_causes)}

    Resolution Steps:
    {' '.join(f'- {step}' for step in resolution_steps)}
    """

    # Generate unique ID
    import uuid
    runbook_id = str(uuid.uuid4())

    # Add to collection
    self.collection.add(
        documents=[doc_text.strip()],
        metadatas=[{
            "title": title,
            "symptom": symptom,
            "category": category,
            "severity": severity,
            "id": runbook_id
        }],
        ids=[runbook_id]
    )

    print(f"Added new runbook: {title}")
    return runbook_id

def retrieve_runbooks(self, query: str, k: int = 3) -> str:
    """
    Retrieve relevant runbooks based on a query.

    Args:
        query: Query string to search for relevant runbooks
        k: Number of top results to return

    Returns:
        Formatted string of retrieved runbooks
    """
    try:
        # Perform similarity search

```

```

        results = self.collection.query(
            query_texts=[query],
            n_results=min(k, self.collection.count()),
            include=["documents", "metadatas"]
        )

        # Format results
        if len(results['documents'][0]) > 0:
            formatted_results = []
            for i, doc in enumerate(results['documents'][0]):
                metadata = results['metadatas'][0][i]
                formatted_result = f"""
Runbook: {metadata['title']}
Symptom: {metadata['symptom']}
Category: {metadata['category']}
Severity: {metadata['severity']}

{doc}

            """.strip()
                formatted_results.append(formatted_result)

            return "\n\n---\n\n".join(formatted_results)
        else:
            return "No relevant runbooks found for the given query."

    except Exception as e:
        print(f"Error retrieving runbooks: {e}")
        return "Error retrieving runbooks."

def get_all_runbooks_metadata(self) -> List[Dict]:
    """
    Get metadata for all runbooks in the knowledge base.

    Returns:
        List of runbook metadata dictionaries
    """
    try:
        all_docs = self.collection.get(include=["metadatas"])
        return all_docs["metadatas"]
    except Exception as e:
        print(f"Error getting runbooks metadata: {e}")
        return []

# Global instance
knowledge_base = None

def get_knowledge_base():
    """Get or create global knowledge base instance."""
    global knowledge_base
    if knowledge_base is None:
        knowledge_base = KnowledgeBase()
    return knowledge_base

def retrieve_runbooks(query: str, k: int = 3) -> str:
    """
    Retrieve relevant runbooks for a query using the knowledge base.

    Args:
        query: Query string to search for relevant runbooks
        k: Number of top results to return

    Returns:
        Formatted string of retrieved runbooks
    """
    kb = get_knowledge_base()
    return kb.retrieve_runbooks(query, k)

def create_knowledge_base(persist_directory: str = "./chroma_db") -> KnowledgeBase:
    """
    Create and initialize a new knowledge base.

    Args:
        persist_directory: Directory to store ChromaDB data

    Returns:
        Initialized KnowledgeBase instance
    """

```

```

"""
return KnowledgeBase(persist_directory)

# Example usage
if __name__ == "__main__":
    # Test the knowledge base
    kb = create_knowledge_base()

    # Test retrieval
    query = "Router CPU utilization going high"
    result = retrieve_runbooks(query, 2)
    print("Retrieved runbooks:")
    print(result)

def _create_sample_runbooks(self) -> List[Dict[str, Any]]:
    """Create 6-8 sample runbooks for the telecom NOC context."""
    return [
        {
            "id": "bgp_peer_down",
            "title": "BGP Peer Down",
            "symptom": "BGP peer goes down",
            "possible_causes": [
                "Physical link failure",
                "Configuration mismatch",
                "Resource exhaustion on router"
            ],
            "resolution_steps": [
                "Check physical connectivity",
                "Verify BGP configuration",
                "Monitor resource usage",
                "Clear BGP session if appropriate"
            ],
            "category": "network",
            "severity": "critical"
        },
        {
            "id": "router_reboot",
            "title": "Router Reboot Required",
            "symptom": "Router unresponsive or crashing",
            "possible_causes": [
                "Software bug",
                "Resource exhaustion",
                "Hardware failure"
            ],
            "resolution_steps": [
                "Check system logs",
                "Verify uptime and resource usage",
                "Perform graceful reboot",
                "Document the event"
            ],
            "category": "hardware",
            "severity": "critical"
        },
        {
            "id": "packet_loss",
            "title": "Packet Loss Detected",
            "symptom": "High packet loss on network links",
            "possible_causes": [
                "Link congestion",
                "Interface hardware issues",
                "Routing problems"
            ],
            "resolution_steps": [
                "Check interface statistics",
                "Monitor traffic patterns",
                "Restart interface if needed",
                "Investigate underlying routing issues"
            ],
            "category": "network",
            "severity": "major"
        },
        {
            "id": "link_down",
            "title": "Link Down",
            "symptom": "Network link goes down",
            "possible_causes": [

```

```

        "Physical cable disconnection",
        "Interface hardware failure",
        "Power outage"
    ],
    "resolution_steps": [
        "Check physical connections",
        "Verify interface status",
        "Restart interface if needed",
        "Check for hardware failures"
    ],
    "category": "hardware",
    "severity": "major"
},
{
    "id": "cpu_utilization_high",
    "title": "High CPU Utilization",
    "symptom": "Router CPU utilization exceeds threshold",
    "possible_causes": [
        "CPU-intensive process",
        "Memory leak",
        "Network traffic spike"
    ],
    "resolution_steps": [
        "Identify high CPU processes",
        "Monitor memory usage",
        "Clear cache if applicable",
        "Consider load balancing"
    ],
    "category": "capacity",
    "severity": "major"
},
{
    "id": "memory_exhaustion",
    "title": "Memory Exhaustion",
    "symptom": "Router memory usage near capacity",
    "possible_causes": [
        "Memory leak in software",
        "Insufficient memory allocation",
        "Large traffic bursts"
    ],
    "resolution_steps": [
        "Check memory usage patterns",
        "Restart memory-hungry processes",
        "Clear buffers and caches",
        "Consider hardware upgrade"
    ],
    "category": "capacity",
    "severity": "major"
},
{
    "id": "bgp_session_flap",
    "title": "BGP Session Flapping",
    "symptom": "BGP session repeatedly goes up and down",
    "possible_causes": [
        "Unstable link",
        "Configuration inconsistency",
        "Timing issues in session negotiation"
    ],
    "resolution_steps": [
        "Check link stability",
        "Verify BGP configurations",
        "Adjust session timers if needed",
        "Clear sessions and monitor"
    ],
    "category": "network",
    "severity": "minor"
},
{
    "id": "dhcp_server_outage",
    "title": "DHCP Server Outage",
    "symptom": "DHCP service unavailable for clients",
    "possible_causes": [
        "DHCP service crash",
        "Network connectivity issues",
        "Configuration errors"
    ],
    "resolution_steps": [
        "Restart DHCP service",
        "Verify network connectivity",
        "Check configuration for errors"
    ],
    "category": "network",
    "severity": "major"
}
]

```

```

        "resolution_steps": [
            "Restart DHCP service",
            "Check network connectivity",
            "Verify configuration files",
            "Monitor lease assignments"
        ],
        "category": "software",
        "severity": "minor"
    }
]

def _initialize_database(self):
    """Initialize the database with sample runbooks."""
    try:
        # Check if collection is empty
        if self.collection.count() == 0:
            print("Initializing knowledge base with sample runbooks...")

            # Convert runbooks to text format for indexing
            documents = []
            metadatas = []
            ids = []

            for runbook in self.sample_runbooks:
                # Create a rich text document combining all information
                doc_text = f"""
                Title: {runbook['title']}
                Symptom: {runbook['symptom']}
                Category: {runbook['category']}
                Severity: {runbook['severity']}

                Possible Causes:
                {' '.join(f'- {cause}' for cause in runbook['possible_causes'])}

                Resolution Steps:
                {' '.join(f'- {step}' for step in runbook['resolution_steps'])}
                """

                documents.append(doc_text.strip())
                metadatas.append({
                    "title": runbook['title'],
                    "symptom": runbook['symptom'],
                    "category": runbook['category'],
                    "severity": runbook['severity'],
                    "id": runbook['id']
                })
                ids.append(runbook['id'])

            # Add documents to ChromaDB
            self.collection.add(
                documents=documents,
                metadatas=metadatas,
                ids=ids
            )
            print(f"Added {len(self.sample_runbooks)} sample runbooks to knowledge base")
        else:
            print(f"Knowledge base already initialized with {self.collection.count()} runbooks")

    except Exception as e:
        print(f"Error initializing knowledge base: {e}")

def add_runbook(self, title: str, symptom: str, possible_causes: List[str],
                resolution_steps: List[str], category: str, severity: str) -> str:
    """
    Add a new runbook to the knowledge base.

    Args:
        title: Runbook title
        symptom: Problem symptom
        possible_causes: List of possible causes
        resolution_steps: List of resolution steps
        category: Runbook category (network, hardware, etc.)
        severity: Severity level

    Returns:
        ID of the added runbook
    """

```

```

"""
# Create document text
doc_text = f"""
Title: {title}
Symptom: {symptom}
Category: {category}
Severity: {severity}

Possible Causes:
{' '.join(f'- {cause}' for cause in possible_causes)}

Resolution Steps:
{' '.join(f'- {step}' for step in resolution_steps)}
"""

# Generate unique ID
import uuid
runbook_id = str(uuid.uuid4())

# Add to collection
self.collection.add(
    documents=[doc_text.strip()],
    metadatas=[{
        "title": title,
        "symptom": symptom,
        "category": category,
        "severity": severity,
        "id": runbook_id
    }],
    ids=[runbook_id]
)

print(f"Added new runbook: {title}")
return runbook_id

def retrieve_runbooks(self, query: str, k: int = 3) -> str:
"""
Retrieve relevant runbooks based on a query.

Args:
    query: Query string to search for relevant runbooks
    k: Number of top results to return

Returns:
    Formatted string of retrieved runbooks
"""
try:
    # Perform similarity search
    results = self.collection.query(
        query_texts=[query],
        n_results=k,
        include=["documents", "metadatas"]
    )

    # Format results
    if len(results['documents'][0]) > 0:
        formatted_results = []
        for i, doc in enumerate(results['documents'][0]):
            metadata = results['metadatas'][0][i]
            formatted_result = f"""
Runbook: {metadata['title']}
Symptom: {metadata['symptom']}
Category: {metadata['category']}
Severity: {metadata['severity']}

{doc}

            """.strip()
            formatted_results.append(formatted_result)

        return "\n\n---\n\n".join(formatted_results)
    else:
        return "No relevant runbooks found for the given query."

except Exception as e:
    print(f"Error retrieving runbooks: {e}")
    return "Error retrieving runbooks."

```



```

def get_all_runbooks_metadata(self) -> List[Dict]:
    """
    Get metadata for all runbooks in the knowledge base.

    Returns:
        List of runbook metadata dictionaries
    """
    try:
        all_docs = self.collection.get(include=["metadatas"])
        return all_docs["metadatas"]
    except Exception as e:
        print(f"Error getting runbooks metadata: {e}")
        return []

# Global instance
knowledge_base = None

def get_knowledge_base():
    """Get or create global knowledge base instance."""
    global knowledge_base
    if knowledge_base is None:
        knowledge_base = KnowledgeBase()
    return knowledge_base

def retrieve_runbooks(query: str, k: int = 3) -> str:
    """
    Retrieve relevant runbooks for a query using the knowledge base.

    Args:
        query: Query string to search for relevant runbooks
        k: Number of top results to return

    Returns:
        Formatted string of retrieved runbooks
    """
    kb = get_knowledge_base()
    return kb.retrieve_runbooks(query, k)

def create_knowledge_base(persist_directory: str = "./chroma_db") -> KnowledgeBase:
    """
    Create and initialize a new knowledge base.

    Args:
        persist_directory: Directory to store ChromaDB data

    Returns:
        Initialized KnowledgeBase instance
    """
    return KnowledgeBase(persist_directory)

# Example usage
if __name__ == "__main__":
    # Test the knowledge base
    kb = create_knowledge_base()

    # Test retrieval
    query = "Router CPU utilization going high"
    result = retrieve_runbooks(query, 2)
    print("Retrieved runbooks:")
    print(result)

```

runtime_settings.json

```
{
  "sla_seconds": 240,
  "notification_recipients": [
    {
      "team": "On-call Network Engineer",
      "channel": "page",
      "contact": "oncall@telco.example"
    },
    {
      "team": "NOC Tier-2",
      "channel": "slack",
      "contact": "#noc-tier2"
    },
    {
      "team": "Engineering Manager",
      "channel": "email",
      "contact": "eng-manager@telco.example"
    }
  ]
}
```